

Charleston Southern University

The Google Abuser

Methodology

By: Michia Rohrssen

Advisor: Dr. Lin

11

Contents

Introduction	2
What is the main goal of The Google Abuser?	2
Don't SEO services already exist? Why can't we do this already?	2
Just how many ranking factors are there?	2
Why The Google Abuser is different.....	3
Just what exactly Does The Google Abuser Do?	3

Introduction

What is the main goal of The Google Abuser?

The main goal of The Google Abuser is to reverse engineer the rankings of a specific keyword and provide suggestions on how to increase the rankings of your website. This has many commercial and personal applications. I have personally used the research and strategies this tool generated to achieve hundreds of front page rankings for clients, which in turn resulted in hundreds of thousands of dollars of revenue.

Don't SEO services already exist? Why can't we do this already?

Well, SEO services certainly exist, but good SEO services are extremely costly. This is because most SEO services work by blindly seeking more backlinks, with very little care given to the specific type of backlinks they need to rank well. Unfortunately blindly increasing ranking factors with no specific strategy means that you have to increase the volume, and hope for the best. This drives costs up and is far from the most efficient method.

Just how many ranking factors are there?

For a while the common number for the industry was 200. This is what Google stated they used, and since Google is at the forefront of search this is what was considered the best and most extensive ranking algorithm. However, in October 2010 Bing announced they were now using more than 1,000 factors to rank webpages in search results. Never one to settle for second place, Google released new data that suggested that the 200 factors was an old figure,

and newer rankings actually used nearly 10,000 factors to rank webpages. With 10,000 ranking factors, SEO can obviously be an extremely overwhelming process. Luckily, with The Google Abuser we don't have to worry about any of that. Source: <http://searchengineland.com/bing-10000-ranking-signals-google-55473>

Why The Google Abuser is different

The Google Abuser is different for many reasons. First, the research that has gone into this program is extensive and thorough. I began researching SEO in 2008 and also began developing websites and boosting their rankings on Google. Over the years I performed many different SEO experiments costing me thousands of dollars. Of course, all these expenses were not wasted, as I have recouped my costs many times over through working for different clients and even my full time job as the director of search marketing for one of the nation's largest Law Firms.

Second, the mathematics behind the program. Rather than relying on guess work, The Google Abuser was built from the ground up to allow proven statistical analysis of a sites rankings. Although this method is more complicated and less user friendly than simply telling the user what to improve, it allows for more flexibility, control and transparency in the analysis.

Just what exactly does The Google Abuser do?

First, The Google Abuser pulls the Titles and URL's of the top X search results from Google, where X is a number from 1-100 entered by the user, along with the users URL and Title. Next, The Google Abuser calculates the 11 key ranking factors which I identified in my research for all sites.

Next, the mean is calculated for each metric. Then, the Pearson's correlation coefficient is calculated for each metric. These metrics are then displayed in a nice table for the user to review and analyze. It also displays key metrics: Page mozRank, Page mozTrust, Domain mozRank and Domain mozTrust on a nice chart so that the user can analyze the strength of the page and domain ranking in the top results. Finally, suggestions are made on which metrics the user should improve to increase rankings. The user is also given the option to export to Excel for further manipulation and review.

The export of this program can be deceptively simple. After all, it is merely 11 metrics for X results. However, the code behind this program is extensive, and takes up more than 1800 lines. A lot of hard work was put into making this program as simple and user friendly as possible, while still containing all the data necessary for power users to get serious use out of the program.

The Google Abuser Manual

By: Michia Rohrsen



Advisor: Dr. Lin

Contents

Chapter 1: Introduction	2
Background of This Tool.....	2
The Purpose of This Tool.....	2
Limitations of This Tool	2
About the Author	3
Chapter 2: The Basics.....	4
Basic Operations.....	4
The Data Table	4
The Page and Domain Strength Graph.....	5
The Recommendations Section.....	6
Chapter 3: Using the Tool	8
Setting up Your Project.....	8
Number of Search Results to Pull.....	8
Your Targeted Keyword.....	8
The Page you Want to Rank	9
Exporting the Data.....	9

Chapter 1: Introduction

Background of This Tool

This tool was developed through years of researching SEO algorithms and my personal work as an SEO professional. Over the years I have personally spent thousands of dollars on SEO experiments. Over 500 front page rankings on Google have been achieved using this research and knowledge. In the end, I was able to identify 11 key factors that when combined can be used to explain the majority of search rankings.

The Purpose of This Tool

This tool is designed to assist SEO professionals in the promotion of websites in any niche. The tool is able to analyze any niche, and give you direct suggestions on how to improve your website rankings.

Limitations of This Tool

Although the tool is very user friendly, the vast field of SEO is a complicated and highly competitive field. Because of this, despite its simple appearance, this software can be very overwhelming for someone new to the world of SEO. Furthermore, although we are able to access certain databases to gain the data required to compute these 11 metrics, there are some metrics which we cannot hope to gather. For example, some of the sites that rank in the top 10 may have unusually high metrics, which indicate they should be ranking higher than they are. In cases like these, the search engines may be filtering out some of the site backlinks due to spam. However, this spam data is not publically accessible, so it is up to the user to identify these cases and adjust their interpretations accordingly.

This tool also assumes that your site has on-page SEO taken care of, and deals only with off-page SEO. If you are not sure if your on-page SEO is correct, a simple Google search for "On-Page SEO" will teach you everything you need to know.

Also, this tool only uses Google search results to analyze data. To the uninitiated this may seem like a limiting factor, but any SEO professional will tell you that this is in fact fine. There are many instances where you may rank for a keyword on Yahoo or Bing but not Google. This is usually because Google has a more sophisticated algorithm. However, it is more important to rank on Google because the majority of search traffic comes from Google. Furthermore, if your website ranks well on Google it will usually rank well on the other search engines.

About the Author

This document and program were created by Michia Rohrsen. He graduated Charleston Southern University graduate with a B.S. in computer science in 2011. If you would like to speak to him about this project you can reach him by email at michiarohrsen@gmail.com

Chapter 2: The Basics

In this chapter we'll cover the basic terms you should know, as well as covering the basic operations of the program. This chapter is NOT designed to teach you the art of SEO. If you would like to learn more about SEO and the methodologies behind this program, there are many free resources online such as SEOmoz and SEOBook.

Basic Operations

When you first open up the program, you will see the following screen. Note that the user interface is deceptively simple, as the program is actually quite complex:

The Google Abuser

Number of Results to Pull: 10 Your Targeted Keyword: The Page You Want to Rank: www.example.com/

Later, you will learn how to use the program, but for now let's just look at a sample output.

The Data Table

Once you enter in your data and hit search, you will see the following screen(see next page)

The Google Abuser

Number of Results to Pull: 5 Your Targeted Keyword: accutane The Page You Want to Rank: www.motleyrice.com Search

Position	Title	URL	Page Total Links	Page External Links	Page Authority	Page mozRank	Page mozTrust	Domain Authority	Domain Total Links	Domain External Links	Domain mozRank	Domain mozTrust	Anchor Links
1	Isotretinoin - Wikip...	en.wikipedia.org/wiki/Isotretinoin	602	455	71.948	4.409	5.796	98.496	280815521	132832624	8.851	8.827	106
2	Accutane Information ...	www.drugs.com/accutane.html	3173	2510	62.903	5.577	5.950	80.914	10808474	446064	6.672	6.786	2247
3	Accutane(Isotretinoin)	www.rxlist.com/accutane-drug.html	3046	2010	62.895	5.559	5.965	84.126	3408758	446601	6.568	6.827	1574
4	IsAccutane/Retaccutan...	www acne.org/accutane.html	5690	48	51.947	5.149	5.795	61.232	651066	18688	5.186	5.412	42
5	Accutane Reviews - Re...	www.accutane-reviews.com	129	118	41.546	4.041	5.467	30.221	216	119	2.932	2.887	116
Your Site	Motley Rice - Autom...	www.motleyrice.com	4460	3275	59.085	5.377	5.912	51.706	49158	6790	4.750	5.031	0
		Mean	252	1028.2	58.248	4.947	5.795	70.998	591	26748819.2	6.042	6.148	817
		Correlation	0.110	-0.431	-0.967	-0.265	-0.641	-0.936	-0.729	-0.709	-0.967	-0.956	-0.336

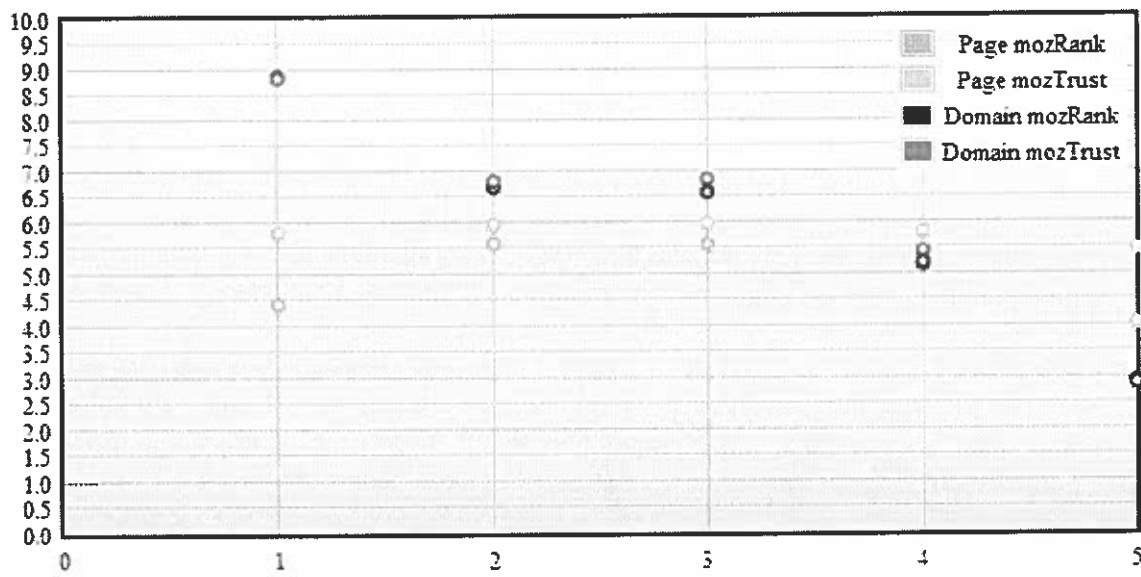
Export to Excel

In this example we've set the number of results to pull to 5, the targeted keyword to "accutane" and we've entered www.motleyrice.com as our website.

The 11 metrics are then pulled for each of the top 5 search results on Google, and also www.motleyrice.com. They are displayed in a nice table so the user can analyze the data quickly and efficiently. The program also calculates the means and correlations of each metric(excluding the users site) so the user can see which metrics are playing the biggest roles on rankings. In this case, the smaller the correlation the stronger the influence(i.e. -1 is the strongest correlation and 1 is the weakest correlation). This is because the best position is 1, not 5 in our search results.

The Page and Domain Strength Graph

Below this table, there is a graph to help the user visualize the strength of the page and domains ranking.



This chart displays the websites rank on the x-axis, and the value for that specific rankings metric on the y-axis. Here you can see a strong correlation between the strength of the domain and the ranking on Google. This makes sense, since our table indicated these had very high correlations.

The Recommendations Section

Finally, below the table there is a section which will give the user recommendations on how to improve their website by telling them which metrics they need to increase to increase rankings. Teaching the user how to improve these metrics is outside the scope of this project since this tool is designed for SEO professionals who will know how to increase these metrics.

How to Improve Your Rankings:

Your biggest weakness is the total number of links containing your targeted keyword. To increase rankings, you should increase the total number of links containing your targeted keyword.

Your second biggest weakness is the total number of external links to your domain. To increase rankings, you should increase the total number of external links to your domain.

Your third biggest weakness is the domain authority of your domain. To increase rankings, you should increase the domain authority of your domain.

Here, the three biggest weaknesses of your site are shown, and recommendations on how to improve your rankings are given.

Chapter 3: Using the Tool

Now that we've gone over everything the tool actually does, let's take a look at how to use the tool and how to get that nifty data for our own keywords and websites!

Setting up Your Project

Number of Search Results to Pull

There are three variables that you need to setup for your project. The first is the number of search results to pull. The databases the program needs to access have strict rate limiting, so the program gets much slower at high numbers. For example, a setting of 2 may take 10 seconds, and a setting of 5 may take 25 seconds, but a setting of 10 can take a few minutes due to the rate limiting. The software has built in measures to deal with this, but it is something you should keep in mind. For quick projects a setting of 5 is acceptable, and 10 should be used when you need a more in depth analysis.

Whatever number you enter will be used to determine how many search results to pull from Google. The software is also able to deal with any Google search modifiers. Any results from Youtube will automatically be filtered out, due to the boost in rankings Google gives these results by default.

Your Targeted Keyword

Next is your targeted keyword. This is the keyword you would like to increase rankings for. Niche research and choosing proper keywords is outside the scope of the project. Once you have determined which keyword you would like to use, enter it in the box so that the analysis can be centered around your keyword.

The Page you Want to Rank

In this box you enter the URL to the page you want to increase rankings for. It's important to remember that Google ranks webpages, not websites. Because of this, you should enter the specific URL that you want to rank, not just the generic URL of your homepage.

Once everything is set, go ahead and click "Search" to begin the process. The program will begin searching and the program will begin pulling the data for analysis.

Exporting the Data

Once you've pulled the results, you may want to store them. This can be useful if you'd like to view the results later to compare your progress or perhaps send the results to another person. To export the results, simply click the "Export to Excel" button. The data will automatically be exported to Excel and opened.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Position	Title	URL	Page Total	Page Exte	Page Auth	Page mozI	Page moz'	Domain A	Domain Tr	Domain E	Domain rr	Domain rr	Anchor Links
2	1	Isotretino	en.wikipe	602	455	71.948	4.409	5.796	98.498	2.81E+08	1.33E+08	8.851	8.827	106
3	2	AccutaneI	www.druj	3173	2510	62.903	5.577	5.95	80.914	10808474	446064	6.672	6.786	2247
4	3	AccutaneI	www.rxlis	3046	2010	62.895	5.559	5.965	84.126	3408758	446601	6.568	6.827	1574
5	4	IsAccutan	www.acni	5690	48	51.947	5.149	5.795	61.232	651066	18688	5.186	5.412	42
6	5	AccutaneI	www.accl	129	118	41.546	4.041	5.467	30.221	216	119	2.932	2.887	116
7	Your Site	Motley Rli	www.mot	4460	3275	59.085	5.377	5.912	51.706	49158	6790	4.75	5.031	0
8		Mean		252	1028.2	58.248	4.947	5.795	70.998	591	26748819	6.042	6.148	817
9		Correlatio		0.11	-0.431	-0.967	-0.265	-0.641	-0.936	-0.729	-0.709	-0.967	-0.956	-0.336



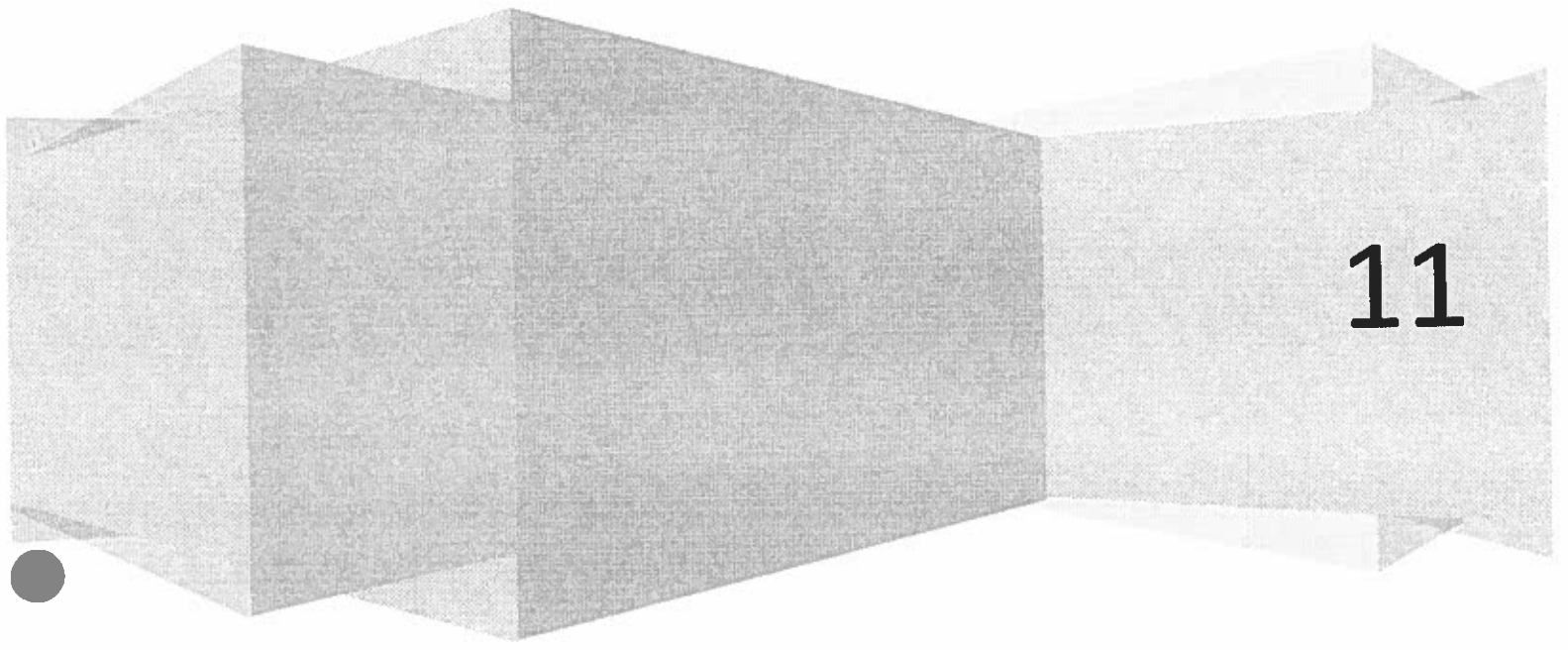
Charleston Southern University

The Google Abuser Developers Manual

By: Michia Rohrssen



Advisor: Dr. Lin



11

Contents

SEOMozAPI.....	2
Introduction	2
AnchorTextService.cs	2
LinksService.cs.....	2
URLMetrics.cs.....	3
Constants.cs	3
Authenticator.cs.....	3
ConnectUtil.cs	3
ProjectTest.....	4
GoogleService.asmx	4
Test.aspx.....	7

SEOMozAPI

Introduction

The SEOMozAPI was developed to allow the user to be able to call the SEOMozAPI from the ASP.NET application. It consists of 6 files; AnchorTextService.cs, Authenticator.cs, ConnectionUtil.cs, Constants.cs, LinkService.cs and URLMetricsService.cs. This code does not live in the project folder, and instead created in a separate project and then brought into the final project as a dll.

AnchorTextService.cs

The anchor text service is designed to assist the user with creating requests to the anchor text service, which is a part of the SEOMozAPI.

`public class AnchorTextResponse` creates a structure that we can use to hold the strings that compose a response from the AnchorText API.

`public class AnchorTextService` main function is to build the request URL, and return a list of anchor texts for the user to review.

LinkService.cs

The links service is designed to assist the user with creating requests to the links service, which is a part of the SEOMozAPI.

`public class LinksResponse` creates a structure that we can use to hold the strings that compose a response from the AnchorText API.

`public class LinkService` main function is to build the request URL, and return a list of link metrics for the user to review.

URLMetrics.cs

The URL metrics service is designed to assist the user with creating requests to the URL Metrics service, which is a part of the SEOmozAPI.

`public class UrlResponse` creates a structure that we can use to hold the strings that compose a response from the URL Metrics API.

`public class UrlMetricsService` main function is to build the request URL, and return a list of URL metrics for the user to review.

Constants.cs

The constants file is designed to make all of the small constants in the program easy to call. Rather than memorizing multiple confusing bit flags for each service, we make it easier for the user by storing them all in one file.

The constants are divided into `URLMetricsConstants`, `AnchorTextConstants` and `LinkConstants`. For more information on why these constants are what they are, visit this page: <http://apiwiki.seomoz.org/w/page/13991145/Request-Response%20Format>

Authenticator.cs

The authenticator file is used to request authentication from the SEOmoz API through HMAC authentication. In order to properly use the authentication, you will need your own API Credentials which can be obtained at: <http://www.seomoz.org/api>

ConnectUtil.cs

Creates a GET HTTP connection to a URL that is passed, and returns the response.

ProjectTest

The ProjectTest folder holds all of the files for the program. This includes the SEOmozAPI.dll, the graphics, the scripts needed, a custom web service for pulling the data, and Test.aspx which is the main file to run the program.

Let's break down the main code files for this project, which are GoogleService.asmx and Test.aspx.

GoogleService.asmx

GoogleService is a webservice that is used to connect to pull Google results and also the SEOmoz Metrics through SEOmozAPI. Let's take a look at each of the functions and see what they do.

`public class SearchResult`

First, we declare the structure SearchResult which will hold all of the results we will need for analysis. It includes the URL, the Title, and the 11 metrics. Note that only anchorlinks must be an int, the rest can be strings.

Next, let's look at the following code:

```
public GoogleService()
{
    //Requests authentication from SEOmoz API, set to expire in 2500 seconds.
    seoMozAuthenticator = new Authenticator("Member-Id", "Secret-Key", 2500);
    //Creates a new URLMetricsService from SEOmoz API, to pull URL Metrics
    Requests
    urlMetricsService = new URLMetricsService(seoMozAuthenticator);
    linkMetricsService = new LinksService(seoMozAuthenticator);
    anchorTextService = new AnchorTextService(seoMozAuthenticator);
}
```

The user will have to provide their own Member-Id and Secret-Key in order for the program to work. The 2500 is how long the authentication is requested to last. Since the data pulling can be quite extensive at higher numbers, we set it to a very high number so the authentication does not time out during data pulling.

```
public string Search(string key, int count, string userUrl)
```

This function pulls search results from Google and returns a list of the search results.

The string "key" comes from the user, and is the targeted keyword. The int count is the number of search results to pull and also comes from the user. Finally, the userUrl is the users website, which is also entered by the user before clicking search.

The code is well commented and explained carefully, but I will give a basic rundown here as well. First, the code makes a queue of the requests. It then pulls the content of the Google search results for your given keyword, and sets it to content. It then passes this content to extract links, and extract links reads the html of the page and pulls out the search results of the page along with the metrics. It does this until it has reached the correct amount of results the user requested. If it runs out of search results on a page, it goes to the next page.

It uses this method because the API's for Google search are depreciated, so we must scrape the search results instead.

Finally, the metrics are also requested for the users URL. Then the array of metrics is passed back. For any search, where X is the count, the array will be X+1 long, and the X+1 spot will be the metrics for the users URL. Now, let's look at the extractLinks function.

```
private void ExtractLinks(string content, ref int currentCount, int maxCount,  
List<SearchResult> list, string key)
```

This function is called by the Search function, and extracts the link out of the current page. It

first makes a list of URL's on the page by matching the html of the page with "<a*class=|>*"

Each one of these <a> tags is a search result.

It then loops through each of the matches and increase the count, creates a new SearchResult, adds the URL and Title and finally pulls the 11 metrics for that URL, then adds it to the SearchResult list. It sleeps .5 seconds between API calls to avoid overloading the API. During this process, any youtube links will be skipped, since Google unfairly boosts the rankings of Youtube videos and it can cause issues with the data that inexperienced SEO professionals may not catch. This process relies on multiple other processes, so let's examine each one.

```
private string GetAttribute(string content, string attrName)
```

This extracts the URL of the content for your search result. The function is coded in a manner that it is able to extract any attribute you tell it to find, but in this case href is passed to it since we need it to extract the URL of the page.

```
private string GetTagContent(string content)
```

This extracts the Title of the content of your search result and returns it as a string. It works like GetAttribute, by matching the content for the title.

```
private SearchResult getURLMetrics(SearchResult data)
```

This function uses the URL Metrics Service we defined earlier to pull 10 of the 11 metrics we need for analysis. Note that the large urlResponse contains all the metrics we need, and the response is then assigned to the different metrics we requested as one comprehensive search result called data.

Unfortunately, the SEOMoz API is woefully unreliable, and often returns incomplete or NULL data. In cases like these, a NullReferenceException will be caught, and the function will sleep for 10 seconds(in case of rate limiting) and then recursively call itself again so that the data can be pulled.

```
private int getAnchorLinks(string url, string key, int count = 0)
```

This function works by finding all of the anchor texts which contain your targeted keyword. For example, if your targeted keyword is "blue widgets" it will count any texts which contain this keyword, so "blue widgets are cool" and "blue widgets for sale" would both be counted. "widgets that are blue" would not be counted, since the keyword does not appear in an exact match, but rather broad match. The total of these matches is counted, and returned. Note that this particular search had a high rate of failure, so a count system is in place to return a 0 if the API call fails 10 times in a row.

```
private string getTitle(string url)
```

This is a one off method that is only needed to pull the Title for the users URL, since this obviously cannot be extracted from Google search results. It calls the URL metrics service, and returns a string that is the Title of the users URL.

That concludes the functions in GoogleService.asmx. Let's take a look at the star of the show, Test.aspx

Test.aspx

The first thing you should notice about this code is the includes. These are important, so make sure you do not change them. The jquery, flot, and excanvas are all used to draw the graph and display data in an attractive fashion.

```
<script type="text/javascript" src="Scripts/jquery.js"></script>
<script type="text/javascript" src="Scripts/jquery.min.js"></script>
<script type="text/javascript" src="Scripts/jquery.flot.js"></script>
<script type="text/javascript" src="Scripts/excanvas.js"></script>
```

Next is the stylesheet. This is not utilized right now, but it is there to make it easy to make stylistic changes in the future.

Now, there are 3 variables that control the display of your data. These are all the length of different metrics. It is important to trim the length, as some of these titles can be extremely long, and the numbers can have infinitely long trailing decimal places. These are given default values, but feel free to change them to whatever you feel is appropriate.

We also create a global meanArray which will hold the mean values of the metrics. It is possible to make this non global, but this method is much cleaner and easier.

The first thing we do is enable or disable the controls, based on the status of the program.

```
function search() {
```

This function is executed when the user clicks "search" or hits enter. It pulls the 3 values the user has entered(count, targeted keyword and user URL) and sends them to the GoogleService web service with this line:

```
ProjectTest.GoogleService.Search(key, count, userUrl, onAjaxSuccess, onAjaxError);
```

During this time, controls will be disabled and the program will display a waiting frame.

If the search succeeds, it will proceed with calculations. If there is an error, the program will alert the user of the error.

```
function onAjaxSuccess(result) {
```

This executes when the search executes successfully. It then begins the calculations. It also switches the wait frame to a new one alerting the user calculations are being performed. Sadly, this happens so quickly it is impossible to notice the switch on a modern computer.

The first thing it does is make an array of the results.

It then calls the following functions in this order:

getRawDataHtmlCode(this creates the table to display results in)

getStatisticsHtmlCode(this calculates the mean and correlation for the metrics, and adds it to the end of the table)

It then adds this data to the Table, and clears the html.

Next, it calculates the weaknesses of your site and builds html for this as well. It then adds this data to the div for the weaknesses.

Finally, it builds a graph to show the strength of the pages and domains that are ranking, so the user can have another way to view the data.

Let's take a look at each of those functions and see what they do.

```
function getRawDataHtmlCode(entryArray) {
```

This function loops through the array passed to it, and creates the table cells to display content to the user in a friendly manner. Long sentences and numbers are trimmed appropriately.

```
function getFixedString(toFixed, fixLen) and function getFixedNum (toFixed, fixLen)
```

Both trim the passed value to the fixLen. The only difference is the method they trim.

Numbers and sentences must be trimmed differently, so they are separate functions.

```
function getStatisticsHtmlCode(entryArray) {
```

This function builds the html code for all of the statistics. First, it builds arrays of each individual array. It makes sure to do this for length-1 results, since it went to the last result, it would also include the users URL in calculations.

It then builds the meanArray using these arrays. It also trims the means so that they display nicely in the table.

Next, it uses the meanArray to build the html code to display means to the user.

Then, it passes the arrays to the `calculateCorrelations` method which calculates the correlations of each array, and returns the correlation. This correlation is trimmed to display properly and added to the table.

```
function calculateMean(dataArray) {  
    Simply calculates the means of the data it is passed. Simple and clean code.
```

```
function calculateCorrelation(dataArray) {  
    This function relies on Pearson's correlation coefficient to calculate the correlation of  
the array it is passed, and the corresponding ranking on Google. Each step is commented, so  
review the comments to understand the computation it is undergoing. The formula it ultimate  
uses is the exact simplified version of Pearson's correlation coefficient.
```

```
function getWeaknesses(dataArray) {  
    This function finds the various weaknesses of the users site by comparing it's metric values to  
the mean of the corresponding results metric. It calculates this first building an array of  
(usersValue/meanValue) – 1. This allows us to sort the values smallest to largest, and pick  
which values have the lowest values. In this case, a smaller value represents a larger weakness.
```

The values are also given names so that the suggestions can be written in plain English. After sorting, the array is sorted biggest weakness to smallest weakness, so picking the first 3 gives you the three biggest weaknesses. This can be modified to do as many weaknesses as you'd like, but 3 is a good start. It then returns the html code to display this nicely in plain English.

```
function createExcelSheet() {  
    This is a simple function that creates a new active X object, which in this case is Excel. It then  
makes a new workbook, and loops through the resultTable rows and simply writes the values to  
the cells in Excel. Simple code, but very useful.
```

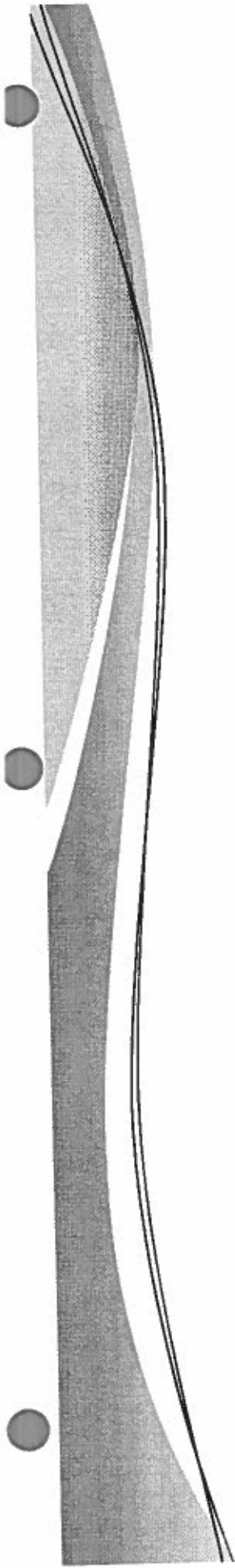
```
function buildGraph(entryArray) {
```

This function builds the nice graph we see. It is based on flot, which is a Javascript graphing library which can be used to make attractive graphs easily and quickly. Make sure you remember your includes!

The first thing we do is setup 4 variables. I ran lots of tests to determine which metrics would be best to graph, and these are the metrics that I believe give the best image of the data. Feel free to modify and test your own though!

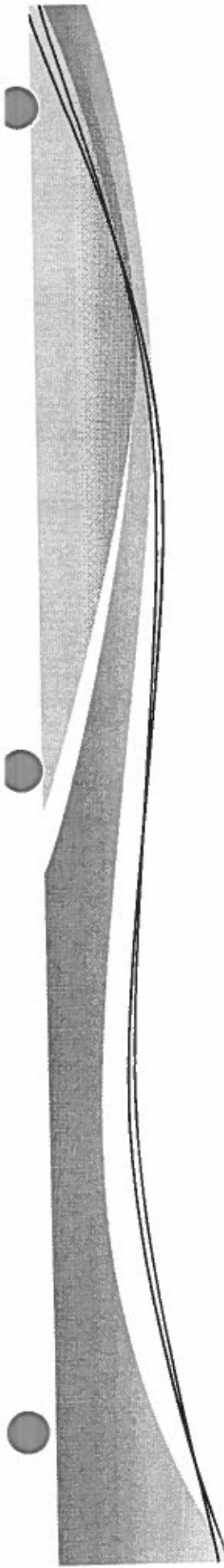
Once the variables are loaded with data, we graph them. Flot supports tons of custom metrics to determine the appearance of the graph, and this is just a small example of what can be done. However, I believe simplicity is best, so I only perform some basic formatting to make it look attractive, sensible, and simple.

Finally, all of this html code is pushed to the divs and tables in the body of the page. This code is fairly self explanatory, and anyone with basic understand of html and CSS should have no problem understanding it. Just know that the ID's are the most important, and the Javascript uses the ID's of each tag to reference, display, and hide data within the area.



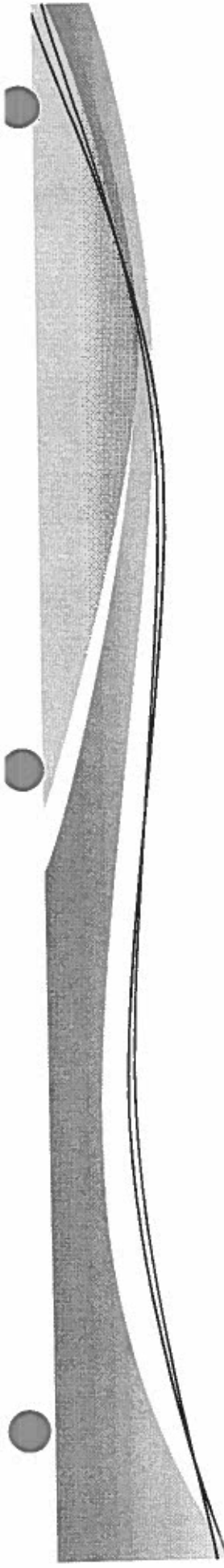
The Google Abuser

By: Michia Rohrsen



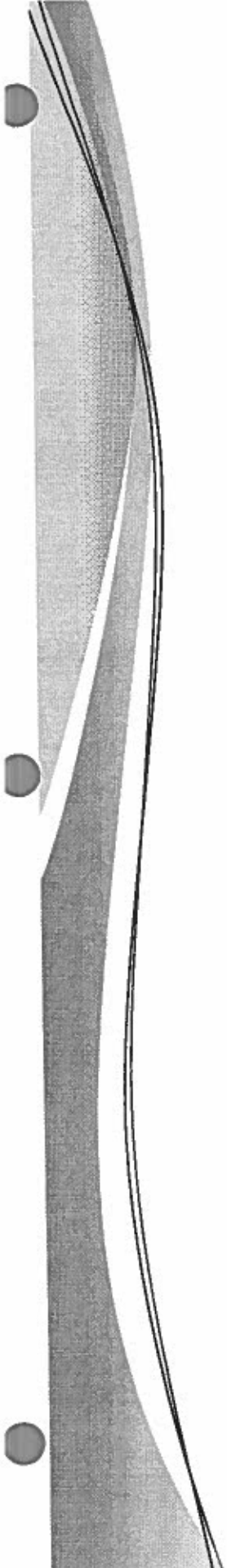
Introduction

- What is the main goal of The Google Abuser?
 - To reverse engineer the rankings of a specific keyword, and provide suggestions on how to increase the rankings of your website.
- Why can't we do this already? Don't SEO services already exist?
 - Of course SEO services exist, but most SEO services are essentially blindly increasing factors without knowing which will have the most impact.



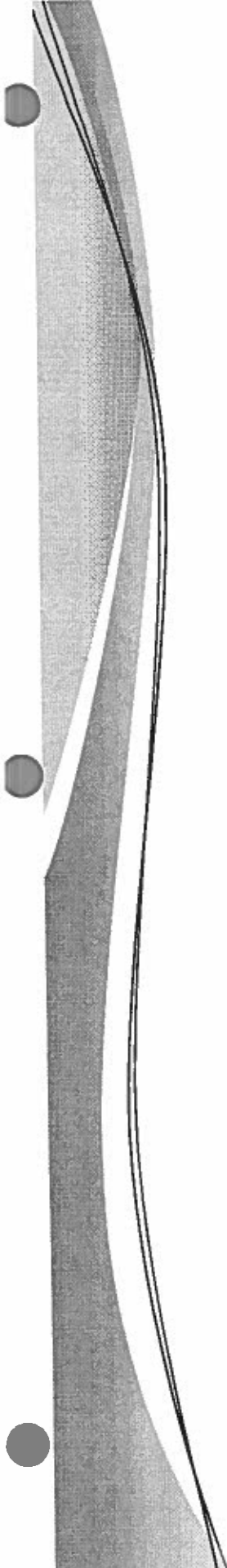
Just how many factors are there?

- For a long time Google stated they used around 200 factors to rank webpages in search results.
- However, in October 2010 Bing announced they were now using more than 1,000 factors to rank webpages in search results.
- Never one to settle for second place, Google announced that the 200 factors was an old figure, and newer rankings actually used nearly 10,000 factors to rank webpages.
- Source: <http://searchengineland.com/bing-10000-ranking-signals-google-55473>



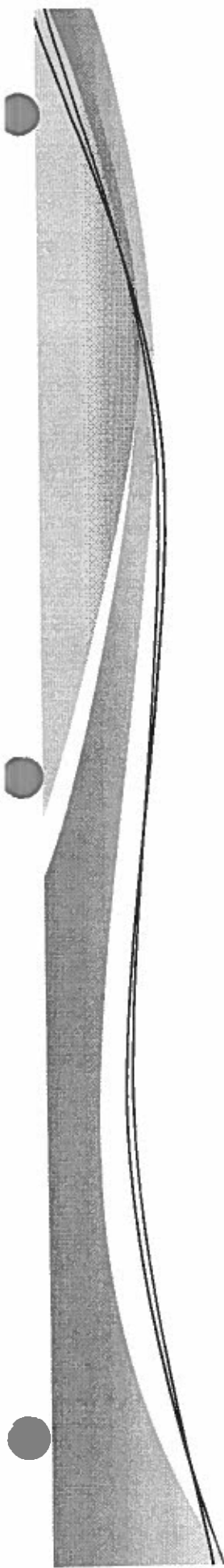
So what do we do?

- Previous SEO efforts essentially relied on increasing factors you believed to be important and hoping your website would increase in rankings.
- However, with 10,000 ranking factors, this obviously isn't a very effective approach.
 - If it was easy, SEO wouldn't be so expensive!



The Google Abuser Difference!

- Why is mine better?
 - First, the research.
 - Since 2008 I've been researching which ranking factors affect rankings the most.
 - I was able to identify 11 key factors to rank websites, and a handful of other factors which boost these 11 key factors.
 - Second, the mathematics.
 - Rather than relying on guess work, my program uses proven statistical methods to determine what metrics are affecting a webpages ranking.

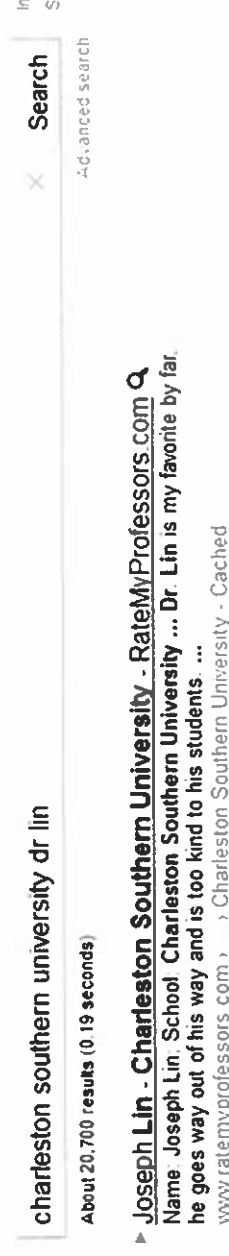


What The Google Abuser Does

- First, it pulls the top x results from Google and the URL of the visitors site.
- Next, it calculates the 11 key ranking factors identified in my research for all sites.
- Then, it calculates the means of each metric.
- Next, it calculates the Pearson's correlation coefficient of each metric.
- It then displays these metrics in a nice table for the user to review.
 - The user has the option to export the data to excel for later review and further manipulation
- Page mozRank, Page mozTrust, Domain mozRank and Domain mozTrust are then graphed on a nice chart.
- Finally, suggestions are made on which metrics the user should improve to increase rankings.

A Note on Case Studies

- Many people often oversimplify the complexity of a full blown SEO campaign for a competitive term.
- For example, “Charleston Southern Dr. Lin” has ratemyprofessors as #1. Does this mean they had an SEO campaign to achieve this?
- No!



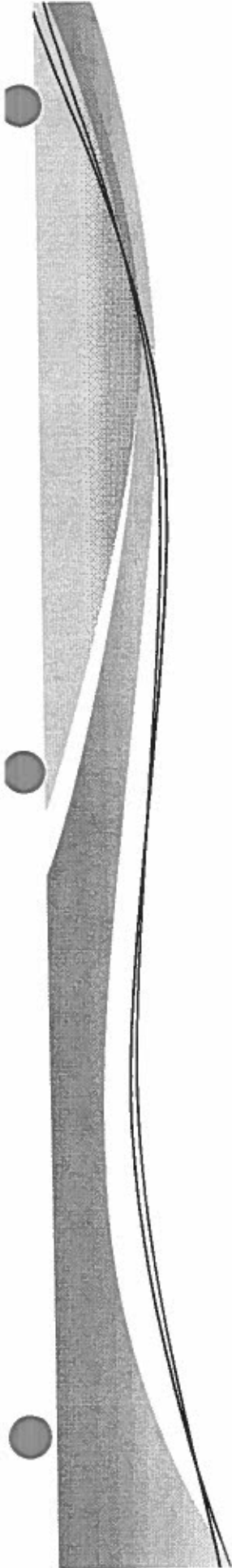
- There are only 20,700 results!
- This site is ranking on pure relevance

A Note on Case Studies(continued)

- How does this compare to the case studies for this project?
- These terms are much more competitive:



- In a case like this, for Google to choose your specific site to be #1 out of 74,400,000 results, you must have a full blown SEO campaign.
- This market is 359,320% more competitive than the previous example. Absolutely no comparisons can be drawn between the SEO of the sites.

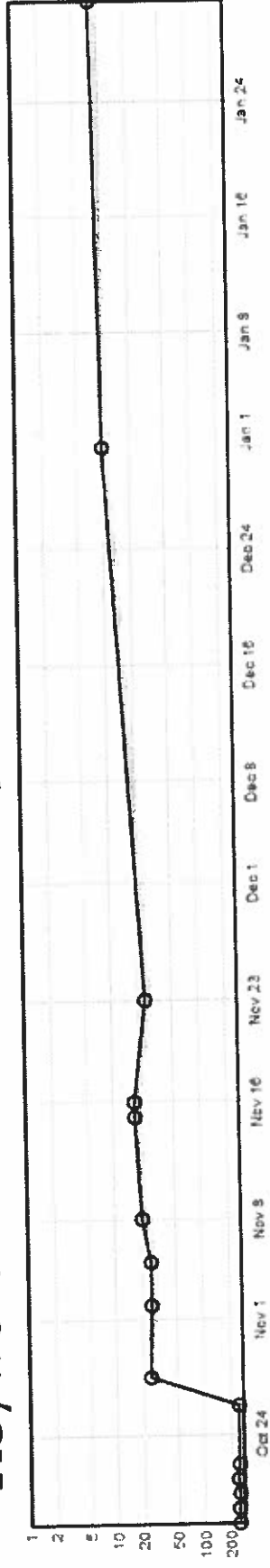


A Case Study

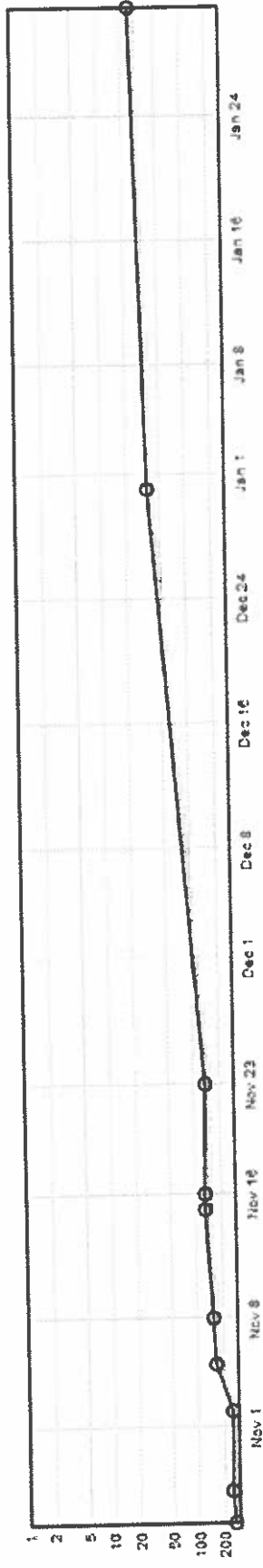
- Brand new site. HIGHLY competitive niche.
- Keyword #1:
 - I noticed a high correlation for page mozRank, domain mozTrust and page external links.
 - So, I increased these metrics.
- Keyword #2:
 - I noticed a high correlation for page mozRank, domain mozTrust, domain external links, and anchor links.
 - Again, I increased these metrics.

Case Study Continued

- What kind of results did my client see from this?
- Keyword 1. Initial Rank: 251. Final rank: 8

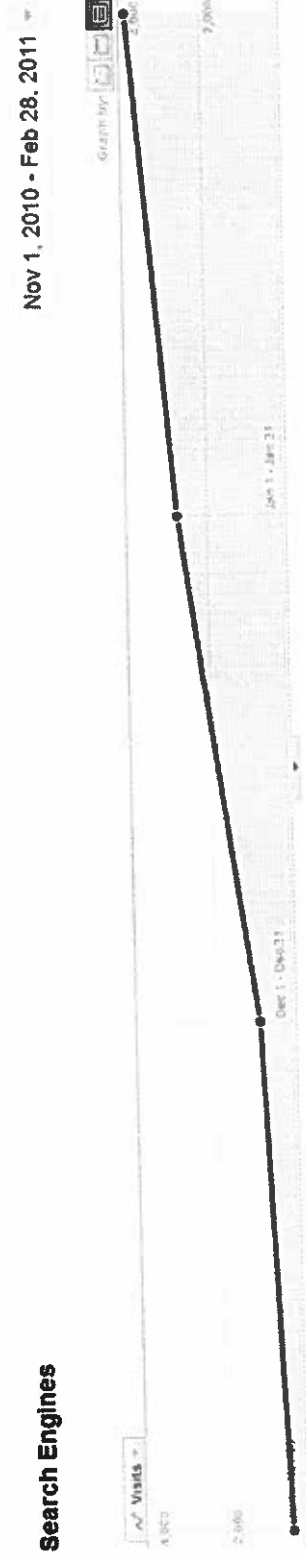


- Keyword 2: Initial rank 267. Final rank: 25

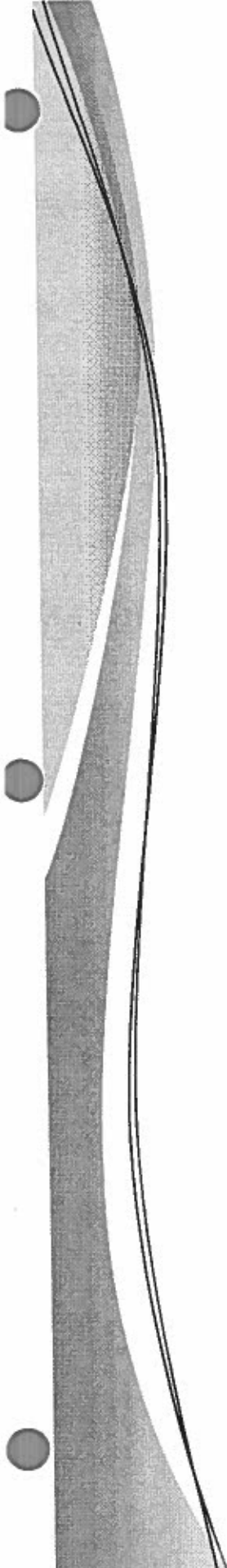


Case Study Results

- We saw a 1700% increase in traffic in 3 months!



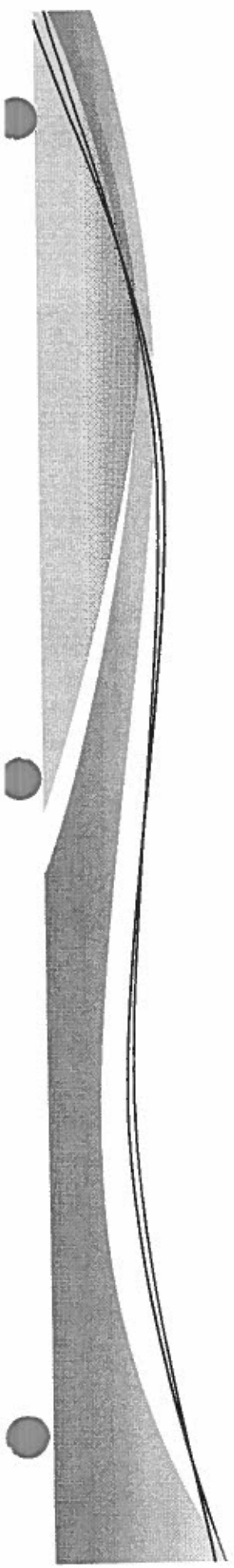
- In this highly competitive niche, each visitor is worth roughly \$2.00 of profit.
- These rankings alone will generate on average \$96,000 profit per year for this company. All done in 3 months.



Further Development

- Outlier detection and calculation adjustments.
 - Initial testing was promising, but also proved to be more complex than initially assumed.
- Multivariable regression of the top 5 ranking factors as determined by strongest correlations.
 - Consulted with Dr. Johnson on the possibility of this. He assured me I would need at least a masters in statistics to do this for my program. I am consulting with a third party statistician, but it will take some time.
- More graphing options for the user.
- A few more custom metrics that require more testing to determine if they are worth adding.
 - Partial match anchor links, on-page SEO analysis and more...

Questions?



The Google Abuser Case

Studies

By: Michia Rohrssen



Advisor: Dr. Lin

Contents

A Note on Case Studies.....	2
Case Study #1.....	3
Overview.....	3
Results.....	4
Case Study #2.....	5
Overview.....	5
Results.....	7
Case Study #3.....	8
Overview.....	8
Results.....	8
Case Study #4.....	10
Overview.....	10
Results.....	10

A Note on Case Studies

It is often tempting to oversimplify the process of ranking a competitive site as simple, after all many people stumble upon rankings almost accidentally.

For example, the search “Charleston Southern University Dr. Lin” pulls up Dr. Lins rate my professor page as #1:

× Search In S

About 20,700 results (0.19 seconds) Advanced search

► [Joseph Lin - Charleston Southern University - RateMyProfessors.com](#) Q

Name: Joseph Lin; School: Charleston Southern University ... Dr. Lin is my favorite by far. he goes way out of his way and is too kind to his students. ...

[www.ratemyprofessors.com](#) > ... > Charleston Southern University - [Cached](#)

Does this mean that this page has a great SEO campaign behind it? Of course not. In this case, it merely means that this page was the most relevant to my search term. As you can see, there were only 20,700 results, so the term is not very competitive to begin with. No one is competing for this term either.

The case studies you see below are all centered around much more competitive terms. For example, my case study for “How to Punch” was for a very competitive term. Just how competitive is it? Let’s take a look:

× Search In: S

About 74,400,000 results (0.09 seconds) Advanced search

► [How To Punch Harder | How To Punch Faster - HowToPunch.Net](#) Q

The best source on mastering the fine art of **punching**. If you're looking to learn **how to punch**, this is it.

[www.howtopunch.net/](#) - [Cached](#) - [Similar](#)

As you can see, there are 74,400,000 results for this keyword. So obviously ranking for this term is very difficult, and cannot be done by merely having the most relevant page. In this case, whomever has the best SEO will win.

To put it simply, the term I am competing for is 359320% more competitive than a simple term such as “Charleston southern university Dr. Lin.” **There are absolutely no comparisons that can be made from analyzing the two sites to determine the effectiveness of SEO.**

Case Study #1

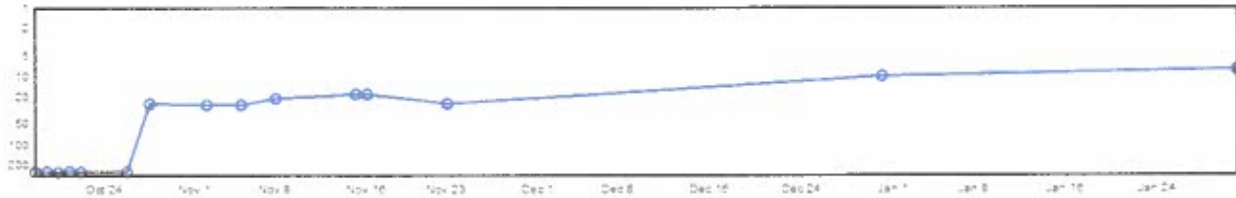
Overview

This was done on a brand new site in a HIGHLY competitive niche. I cannot reveal specific keywords, since this was done for a client of mine.

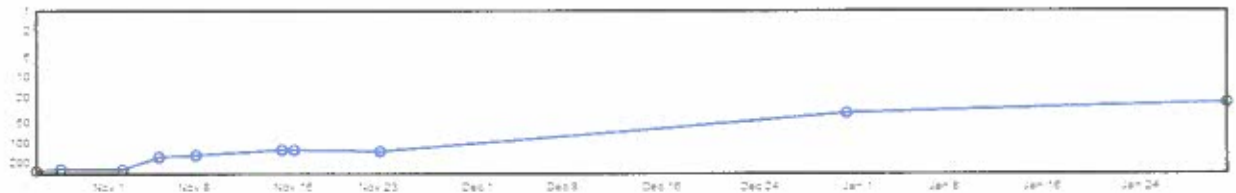
- Keyword #1:
 - I noticed a high correlation for page mozRank, domain mozTrust and page external links.
 - So, I increased these metrics.
- Keyword #2:
 - I noticed a high correlation for page mozRank, domain mozTrust, domain external links, and anchor links.
 - Again, I increased these metrics.

Results

- What kind of results did my client see from this?
- Keyword 1: Initial Rank: 251. Final rank: 8



- Keyword 2: Initial rank 267. Final rank: 25



And what do these ranking increase translate to?

- We saw a 1700% increase in traffic in 3 months!



- In this highly competitive niche, each visitor is worth roughly \$2.00 of profit.
- These rankings alone will generate on average \$96,000 profit per year for this company. All done in 3 months.

Case Study #2

Overview

In June 2010, I had a great opportunity. I was asked to give a presentation to the head of Asbestos litigation at Motley Rice about web marketing and how he could improve their rankings on search engines. In order to properly prepare for this meeting, I did research similar to the calculations this tool does. Unfortunately this was before my tool was fully operational, so it was not quite as in depth as the research this tool is able to perform(although it was more confusing).

Here's a slide from the presentation I gave regarding my research:

Competition Research

- As I began my competition research it became abundantly clear that the majority of the keywords were highly competitive.
- In order to get a better handle on the market, I did very in depth competition research.

Keyword	Searches	SEO Traffic	Adwords CPC	SEO Value	BLD Site 1	BLP Site 1	Link Rel	PR Score	BLD Site 2	BLP Site 2	Link Rel	PR Score	BLD Site 3	BLP Site 3	Link Rel	PR Score
mesothelioma lawyers	1,989	835	\$30	\$25,086	4,670	410	70%	6	1,050	908	30%	2	30,101	266	50%	7
asbestos attorney	1,627	683	\$39	\$26,883	329*	226	20%	1	4,212	3,701	54%	6*	5,210	4,980	50%	6*
mesothelioma lawyer	1,332	559	\$36	\$20,224	1,050	908	20%	2	30,101	266	60%	4	14,800	6,610	15%	5*
mesothelioma attorney	1,332	559	\$38	\$21,393	2,870	787	15%	2	1,050	908	10%	2	10,603	89	30%	8
mesothelioma settlement	891	374	\$29	\$10,804	474	29	100%	0	322	21	90%	0	37	7	15%	0
asbestos lawyer	891	374	\$38	\$14,123	1,480,491	46	60%	1	293,233	57	30%	4	14,800	6,610	54%	5*
mesothelioma attorneys	891	374	\$39	\$14,699	10,603	89	20%	8	1,050	908	10%	2	4,670	410	55%	4
asbestos lawyers	487	205	\$37	\$7,568	1,480,491	46	60%	1	293,233	57	30%	4	749	18	50%	1
asbestos attorneys	398	167	\$38	\$6,374	329*	226	20%	1	6,010	2,030	14%	7*	5,210	4,980	50%	6*
asbestos lawsuit	325	137	\$34	\$4,644	51,613	22	70%	1	4,360	105	40%	3	28	15	5%	0
mesothelioma lawsuit	266	112	\$28	\$3,143	51,613	22	70%	1	4,360	106	50%	3	19,802	23	80%	0
asbestos cancer lawsuit	217	91	\$26	\$2,336	217*	50	40%	1	14,314	21	30%	0	20	10	100%	2

Key:

* = keyword domain name

** = possible inaccurate due to 1000+ backlinks

After I selected my targeted keywords, it was time to see where Motley Rice currently ranked for these keywords. Here is the slide from the presentation I gave:

● These are our current rankings for these terms:

Keyword	Broad Position			Ranking Page	Click down arrows for more results	BL	PR
	G	Y	B				
asbestos lawyer	75	+	+	http://www.motleyrice.com/asbestos-and-mesothelioma-lawyer	21	4	
asbestos lawyers	143	+	+	http://www.motleyrice.com/asbestos-and-mesothelioma-lawyers	21	4	
asbestos lawsuit	219	+	+	http://www.motleyrice.com/asbestos-and-mesothelioma-lawsuit	1	0	
mesothelioma lawsuit	-	-	-	not found	0	0	

In this case a + sign means the search engines know of your site, but you are not in the top 1000 results. A – sign indicates that the search engines do not even consider your site relevant to this ranking. **Motley Rice had been targeting these terms for years, and this was obviously a very competitive market.**

However, I had faith in my research and made a rather bold claim at the end of my presentation. I promised the head of asbestos(who handles all the hiring for the company) that if he could get me a position as the lead web marketing specialist, I could get him top 10 rankings in 30 days or less. It's a bold and scary claim even for me, since these rankings are hard to come by. However, I had faith in the research so I stood by my claim.

He took me up on my offer, and gave me a position. Using my research, I was able to determine exactly the kinds of links we would need to rank #1 for each keyword. It should be noted that although this approach is considered sophisticated by most SEO standards, the tool I developed actually goes much deeper than this. Some of the data is not shown on this slide

because it would confuse the people I was giving the presentation to, and some of the data simply didn't exist yet.

The Actual Numbers

The following links need to be built for #1 rankings(ideally)

- Anchor Text: "Mesothelioma Settlement"
 - 6 High Pr Links
 - 21 Article Links
 - 3 Social Web 2.0 Links
- Anchor Text: "Asbestos Lawyers"
 - 12 High PR Links
 - 42 Article Links
 - 6 Social Web 2.0 Links
- Anchor Text: "Asbestos lawyer"
 - 3 High PR Links
 - 11 Article Links
 - 2 Social Web 2.0 Links
- Anchor Text: "Asbestos Lawsuit"
 - 5 High PR Links
 - 16 Article Links
 - 2 Social Web 2.0 Links
- Anchor Text: "Mesothelioma Lawsuit"
 - 5 High PR Links
 - 16 Article Links
 - 2 Social Web 2.0 Links

Results

At the end of the 30 days, I gave my presentation again, presenting our rankings:

Rankings After 30 Days

- After 30 days, we were able to achieve the following rankings:

	Keyword	Broad Position			Ranking Page	Click down arrows for more results
		G	Y	B +		
☐	asbestos lawsuit	2	2	2	U: http://www.motte-nice.com/asbestos-lawyers	
☐	asbestos lawyers	2	2	2	U: http://www.motte-nice.com/asbestos-lawyers	
☐	mesothelioma lawsuit	7	10	10	U: http://www.motte-nice.com/asbestos-lawyers	
☐	asbestos lawyer	3	14	14	U: http://www.motte-nice.com/asbestos-lawyers	

Case Study #3

Overview

For my personal self defense business, I needed to boost rankings to drive more visitors to my website. I decided to create miniature sites based on high traffic keywords to help capture traffic for these keywords (Note: since the Google 2011 Panda update this is a technique I no longer use).

Results

The first site was designed to rank for the term "how to punch." I analyzed the metrics of my competitors, and increased what the tool suggested. In no time, I was #1.

how to punch



Search

Int
Se

About 74,400,000 results (0.09 seconds)

[Advanced search](#)

► [How To Punch Harder | How To Punch Faster - HowToPunch.Net](#)

The best source on mastering the fine art of **punching**. If you're looking to learn **how to punch**, this is it.

[www.howtopunch.net/](#) - [Cached](#) - [Similar](#)

The second site was designed to rank for the term "fighting techniques." Again, I analyzed the metrics of my competitors, and increased what the tool suggested. In no time, I was #1.

fighting techniques



Search

Int
Se

About 1,300,000 results (0.17 seconds)

[Advanced search](#)

► [Fighting Techniques | Street Fighting Techniques | Self Defense ...](#)

Looking to learn raw street **fighting techniques**? Come inside to **study** and master the latest **brutal fighting techniques**.

[www.fightingtechniques.org/](#) - [Cached](#) - [Similar](#)

Case Study #4

Overview

One of my most recent case studies is actually taking place right now as I type this document. One of my clients requested that I help him create and rank a website for World of Warcraft related terms. On March 24, 2011 the pages we wanted to rank went live.

I used the tool to determine which metrics would increase rankings best, and developed the site accordingly:

How to Improve Your Rankings:

Your biggest weakness is the total number of links to your page. To increase rankings, you should increase the total number of links to your page.

Your second biggest weakness is the total number of external links to your page. To increase rankings, you should increase the total number of external links to your page.

Your third biggest weakness is the mozRank of your page. To increase rankings, you should increase the mozRank of your page.

Results

Following these suggestions, our rankings went from nonexistent to #5 for “wow classes” and #8 for “wow quests” in less than 30 days and rankings continue to increase. You can see proof of the rankings in the screenshots below:

wow classes

About 40,400,000 results (0.11 seconds)

► [Class - WoWWiki - Your guide to the World of Warcraft](#) 🔍

A **class** is the primary adventuring style of a player character which determines the type of weapons and armor it can use, as well as what abilities, powers, ...

[www.wowwiki.com/Class](#) - [Cached](#) - [Similar](#)

[Choosing a class](#)

[Starting a](#)

[Hero class](#)

[More results from wowwiki.com »](#)

[Classes - Game - World of Warcraft](#) 🔍

This new and improved class guide contains more information about each of **World of Warcraft's classes** with a description of their abilities, playstyles, ...

[us.battle.net/wow/en/game/class/](#) - [Cached](#) - [Similar](#)

[WoW Class Quiz - Which World of Warcraft Class Is Best For You?](#) 🔍

Which **World of Warcraft Class** is the best for you? Take the **WOW CLASS QUIZ** and find which **World of Warcraft class** you should be!

[www.quizrocket.com/wow-class-quiz](#) - [Cached](#) - [Similar](#)

[WoW Classes Network](#) 🔍

Jan 17, 2011 ... Tags: [buy wow gold](#), [warcraft gold](#), [world of warcraft classes](#), [world of warcraft gold](#), [world of warcraft trolls](#), [wow classes](#), [wow trolls](#) ...

[wowclasses.net/](#) - [Cached](#) - [Similar](#)

[Choosing a New World of Warcraft Class | WoW Quest Helper Reviews ...](#) 🔍

Mar 24, 2011 ... This also makes the game highly accessible because the different **WoW classes** best suit different playing styles. A character class that best ...

[www.wowquestshelper.com/.../wow-classes/choosing-a-new-world-of-warcraft-class/](#) -

[Cached](#)

[My Clients Site](#)

wow quests

About 6,160,000 results (0.08 seconds)

► **Quests - World of Warcraft**

List of all existing **quests** in **World of Warcraft** ... **Quests - World of Warcraft** Type: clear
Normal, Group, Dungeon, Raid, PvP, Heroic ...

www.wowhead.com/quests - [Cached](#) - [Similar](#)

Wowhead: World of Warcraft Database

A **World of Warcraft (WoW)** database site with items, **quests**, forums, news ...

Talent Calculator - [Quests](#) - [Items](#) - [Archaeology](#)

www.wowhead.com/ - [Cached](#) - [Similar](#)

[+ Show more results from wowhead.com](#)

ZAM World of Warcraft :: A WoW Community and Wiki Site

ZAM World of Warcraft is your ultimate destination for **WoW** information featuring a wiki, strategy guides, forums, chat rooms, images, videos and more.

wow.allakhazam.com/ - [Cached](#) - [Similar](#)

Quest Listing for WOW - World of Warcraft

Quest Listing for WOW - World of Warcraft, **Wrath of the Lich King**, **Burning Crusade**, **WoW**, maps, guides, items, **quests**.

wow.gamepressure.com/quests.asp - [Cached](#) - [Similar](#)

Quest - WoWWiki - Your guide to the World of Warcraft

Feb 3, 2010 ... A **quest** is a task given to a player character that yields a reward when completed. Most **quests** are given by an NPC (non-player character) ...

www.wowwiki.com/Quest - [Cached](#) - [Similar](#)

Beginner's Guide - Game - World of Warcraft

In **World of Warcraft**, you play the role of a fantasy hero. Over the course of his or her life, your character will brave thousands of **quests**, learn new and ...

us.battle.net/wow/game/guide/ - [Cached](#)

Questkeep.com - Quest Guides for World of Warcraft

Database of handwritten **quest** walk-throughs and general tips.

questkeep.com/ - [Cached](#) - [Similar](#)

How to Get Quick Help for your World of Warcraft Quests | WoW ...

Mar 24, 2011 ... **World of Warcraft** features a growing number of **quests** of various categories. These **quests** may range from the ones you do alone to the more ...

www.wowquestshelper.com/.../wow-quests/how-to-get-quick-help-for-your-world-of-warcraft-quests/ - [Cached](#)

My Clients Site

The Google Abuser Code

By: Michia Rohrssen

Advisor: Dr. Lin

Contents

SEOMozAPI.....	2
AnchorTextService.cs.....	2
Authenticator.cs	4
ConnectionUtil.cs.....	6
Constants.cs	7
LinksService.cs	9
URLMetricsService.cs.....	12
ProjectTest	18
GoogleService.aspx.....	18
Test.aspx	24

SEOmozAPI

AnchorTextService.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Web;
using System.Web.Script.Serialization;

namespace SEOmozAPI
{
    public class AnchorTextResponse
    {
        public string apuef;

        public string apuemp;

        public string apuep;

        public string apueu;

        public string apuf;

        public string apuif;

        public string apuimp;

        public string apuii;

        public string aput;

        public override string ToString()
        {
            return "AnchorTextResponse [apuef=" + apuef + ", apuemp=" + apuemp
                + ", apuep=" + apuep + ", apueu=" + apueu + ", apuf=" + apuf
                + ", apuif=" + apuif + ", apuimp=" + apuimp + ", apuii="
                + apuii + ", aput=" + aput + "]";
        }
    }

    public class AnchorTextService
    {
        public Authenticator Authenticator
        {
            get;
            set;
        }

        public AnchorTextService()
        {
        }
    }
}
```



```

/**
 *
 * @param authenticator
 */
public AnchorTextService(Authenticator authenticator)
{
    this.Authenticator = authenticator;
}
/**
 *
 * @param objectURL
 * @param scope
 * @param sort
 * @param col
 * @return a set of anchor text terms of phrases aggregated across links to a page
or domain.
 *
 * @see #GetAnchorText(string, string, string, int, int, int)
 */
public List<AnchorTextResponse> GetAnchorText(string objectURL, string scope,
string sort, long col)
{
    return GetAnchorText(objectURL, scope, sort, col, -1, -1);
}

/**
 * This method returns a set of anchor text terms of phrases aggregated across
links to a page or domain.
 *
 * @param objectURL
 * @param scope determines the scope of the link, and takes one of the following
values:
 * phrase_to_page: returns a set of phrases found in links to the specified page
 * phrase_to_subdomain: returns a set of phrases found in links to the specified
subdomain
 * phrase_to_domain: returns a set of phrases found in links to the specified root
domain
 * term_to_page: returns a set of terms found in links to the specified page
 * term_to_subdomain: returns a a set of terms found in links to the specified
subdomain
 * term_to_domain: returns a a set of terms found in links to the specified root
domain
 * @param sort etermines the sorting of the links, in combination with limit and
offset, this allows fast access to the top links by several orders:
 * domains_linking_page: the phrases or terms found in links from the most number
of root domains linking are returned first
 * @param col determines what fields are returned
 * @param offset The start record of the page can be specified using the Offset
parameter
 * @param limit The size of the page can by specified using the Limit parameter.
 * @return a set of anchor text terms of phrases aggregated across links to a page
or domain.
 */
public List<AnchorTextResponse> GetAnchorText(string objectURL, string scope,
string sort, long col, int offset, int limit)
{
    string urlToFetch = "http://lsapi.seomoz.com/linkscope/anchor-text/" +
HttpUtility.HtmlEncode(objectURL) + "?" + Authenticator.GetAuthenticationStr();

```

```

        if(scope != null)
        {
            urlToFetch = urlToFetch + "&Scope=" + scope;
        }

        if(sort != null)
        {
            urlToFetch = urlToFetch + "&Sort=" + sort;
        }

        if(col > 0)
        {
            urlToFetch = urlToFetch + "&Cols=" + col;
        }

        if(offset >= 0)
        {
            urlToFetch = urlToFetch + "&Offset=" + offset;
        }
        if(limit >= 0)
        {
            urlToFetch = urlToFetch + "&Limit=" + limit;
        }
        string response = ConnectionUtil.MakeRequest(urlToFetch);

        return new
JavaScriptSerializer().Deserialize<List<AnchorTextResponse>>(response);
    }
}

```

Authenticator.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Security.Cryptography;
using System.IO;
using System.Web;

namespace SEOMozAPI
{
    public class Authenticator
    {
        /**
         * AccessID The user's Access ID
         */
        public string AccessID
        {
            get;
            set;
        }
    }
}

```

```

    /**
     * SecretKey The user's Secret Key
     */
    public string SecretKey
    {
        get;
        set;
    }

    /**
     * ExpiresInterval The interval after which the authentication string expires
     * Default 300s
     */
    public long ExpiresInterval
    {
        get;
        set;
    }

    public Authenticator()
    {
        ExpiresInterval = 300;
    }

    /**
     * Constructor to set all the variables
     *
     * @param AccessID
     * @param SecretKey
     * @param ExpiresInterval
     */
    public Authenticator(string accessID, string secretKey, long expiresInterval)
    {
        this.AccessID = accessID;
        this.SecretKey = secretKey;
        this.ExpiresInterval = expiresInterval;
    }

    /**
     *
     * This method calculates the authentication string based on the
     * user's credentials.
     *
     * Set the user credentials before calling this method
     *
     * @return the authentication string
     *
     * @see #setAccessID(string)
     * @see #setSecretKey(string)
     */
    public string GetAuthenticationStr()
    {
        long expires = GetTime() + ExpiresInterval;

        string stringToSign = AccessID + "\n" + expires;
        string urlSafeSignature;

```

```

        try
        {
            ASCIIEncoding encoding = new ASCIIEncoding();
            HMAC Hmac = HMAC.Create("HmacSHA1");
            Hmac.Key = encoding.GetBytes(SecretKey);
            Hmac.Initialize();
            CryptoStream cs = new CryptoStream(Stream.Null, Hmac,
CryptoStreamMode.Write);
            cs.Write(encoding.GetBytes(stringToSign), 0,
encoding.GetBytes(stringToSign).Length);
            cs.Close();
            byte[] rawResult = Hmac.Hash;
            urlSafeSignature =
HttpUtility.UrlEncode(Convert.ToBase64String(rawResult, 0, rawResult.Length));
        }
        catch (Exception e)
        {
            throw new Exception("Failed to generate HMAC : " + e.Message);
        }

        string authenticationStr = "AccessID=" + AccessID + "&Expires=" +
expires + "&Signature=" + urlSafeSignature;

        return authenticationStr;
    }

    private static Int64 GetTime()
    {
        Int64 retval = 0;
        DateTime st = new DateTime(1970, 1, 1);
        TimeSpan t = (DateTime.Now.ToUniversalTime() - st);
        retval = (Int64)(t.TotalSeconds);
        return retval;
    }
}
}

```

ConnectionUtil.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Net;

namespace SEOmozAPI
{
    public class ConnectionUtil
    {
        /**
         *
         * Method to make a GET HTTP connecton to

```

```

    * the given url and return the output
    *
    * @param urlToFetch url to be connected
    * @return the http get response
    */
    public static string MakeRequest(string urlToFetch)
    {
        try
        {
            WebRequest webRequest = WebRequest.Create(urlToFetch);
            System.IO.StreamReader requestStream = new
System.IO.StreamReader(webRequest.GetResponse().GetResponseStream());

            return requestStream.ReadToEnd();
        }
        catch (System.Exception ex)
        {
        }

        return "";
    }
}

```

Constants.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace SEOmozAPI
{
    public class URLMetricsConstants
    {
        public const long URLMETRICS_COL_ALL = 0;
        public const long URLMETRICS_COL_TITLE = 1;
        public const long URLMETRICS_COL_URL = 4;
        public const long URLMETRICS_COL_SUBDOMAIN = 8;
        public const long URLMETRICS_COL_ROOT_DOMAIN = 16;
        public const long URLMETRICS_COL_EXTERNAL_LINKS = 32;
        public const long URLMETRICS_COL_SUBDMN_EXTERNAL_LINKS = 64;
        public const long URLMETRICS_COL_ROOTDMN_EXTERNAL_LINKS = 128;
        public const long URLMETRICS_COL_JUICE_PASSING_LINKS = 256;
        public const long URLMETRICS_COL_SUBDMN_LINKS = 512;
        public const long URLMETRICS_COL_ROOTDMN_LINKS = 1024;
        public const long URLMETRICS_COL_LINKS = 2048;
        public const long URLMETRICS_COL_SUBDMN_SUBDMN_LINKS = 4096;
        public const long URLMETRICS_COL_ROOTDMN_ROOTDMN_LINKS = 8192;
        public const long URLMETRICS_COL_MOZRANK = 16384;
        public const long URLMETRICS_COL_SUBDMN_MOZRANK = 32768;
        public const long URLMETRICS_COL_ROOTDMN_MOZRANK = 65536;
        public const long URLMETRICS_COL_MOZTRUST = 131072;
        public const long URLMETRICS_COL_SUBDMN_MOZTRUST = 262144;
    }
}

```

```

public const long URLMETRICS_COL_ROOTDMN_MOZTRUST = 524288;
public const long URLMETRICS_COL_EXTERNAL_MOZRANK = 1048576;
public const long URLMETRICS_COL_SUBDMN_EXTERNALDMN_JUICE = 2097152;
public const long URLMETRICS_COL_ROOTDMN_EXTERNALDMN_JUICE = 4194304;
public const long URLMETRICS_COL_SUBDMN_DOMAIN_JUICE = 8388608;
public const long URLMETRICS_COL_ROOTDMN_DOMAIN_JUICE = 16777216;
public const long URLMETRICS_COL_CANONICAL_URL = 268435456;
public const long URLMETRICS_COL_HTTP_STATUS_CODE = 536870912;
public const long URLMETRICS_COL_LINKS_TO_SUBDMN = 4294967296L;
public const long URLMETRICS_COL_LINKS_TO_ROOTDMN = 8589934592L;
public const long URLMETRICS_COL_ROOTDMN_LINKS_SUBDMN = 17179869184L;
public const long URLMETRICS_COL_PAGE_AUTHORITY = 34359738368L;
public const long URLMETRICS_COL_DOMAIN_AUTHORITY = 68719476736L;
}

public struct LinksConstants
{
    public const string LINKS_SCOPE_PAGE_TO_PAGE = "page_to_page";
    public const string LINKS_SCOPE_PAGE_TO_SUBDOMAIN = "page_to_subdomain";
    public const string LINKS_SCOPE_PAGE_TO_DOMAIN = "page_to_domain";
    public const string LINKS_SCOPE_DOMAIN_TO_PAGE = "domain_to_page";
    public const string LINKS_SCOPE_DOMAIN_TO_SUBDOMAIN = "domain_to_subdomain";
    public const string LINKS_SCOPE_DOMAIN_TO_DOMAIN = "domain_to_domain";

    public const string LINKS_SORT_PAGE_AUTHORITY = "page_authority";
    public const string LINKS_SORT_DOMAIN_AUTHORITY = "domain_authority";
    public const string LINKS_SORT_DOMAINS_LINKING_DOMAIN =
"domains_linking_domain";
    public const string LINKS_SORT_DOMAINS_LINKING_PAGE = "domains_linking_page";

    public const string LINKS_FILTER_INTERNAL = "internal";
    public const string LINKS_FILTER_EXTERNAL = "external";
    public const string LINKS_FILTER_NOFOLLOW = "nofollow";
    public const string LINKS_FILTER_FOLLOW = "follow";
    public const string LINKS_FILTER_301 = "301";

    public const long LINKS_COL_ALL = 0;
    public const long LINKS_COL_TITLE = 1;
    public const long LINKS_COL_URL = 4;
    public const long LINKS_COL_SUBDOMAIN = 8;
    public const long LINKS_COL_ROOT_DOMAIN = 16;
    public const long LINKS_COL_EXTERNAL_LINKS = 32;
    public const long LINKS_COL_SUBDMN_EXTERNAL_LINKS = 64;
    public const long LINKS_COL_ROOTDMN_EXTERNAL_LINKS = 128;
    public const long LINKS_COL_JUICE_PASSING_LINKS = 256;
    public const long LINKS_COL_SUBDMN_LINKS = 512;
    public const long LINKS_COL_ROOTDMN_LINKS = 1024;
    public const long LINKS_COL_LINKS = 2048;
    public const long LINKS_COL_SUBDMN_SUBDMN_LINKS = 4096;
    public const long LINKS_COL_ROOTDMN_ROOTDMN_LINKS = 8192;
    public const long LINKS_COL_MOZRANK = 16384;
    public const long LINKS_COL_SUBDMN_MOZRANK = 32768;
    public const long LINKS_COL_ROOTDMN_MOZRANK = 65536;
    public const long LINKS_COL_MOZTRUST = 131072;
    public const long LINKS_COL_SUBDMN_MOZTRUST = 262144;
    public const long LINKS_COL_ROOTDMN_MOZTRUST = 524288;
    public const long LINKS_COL_EXTERNAL_MOZRANK = 1048576;
    public const long LINKS_COL_SUBDMN_EXTERNALDMN_JUICE = 2097152;

```

```

        public const long LINKS_COL_ROOTDMN_EXTERNALDMN_JUICE = 4194304;
        public const long LINKS_COL_SUBDMN_DOMAIN_JUICE = 8388608;
        public const long LINKS_COL_ROOTDMN_DOMAIN_JUICE = 16777216;
        public const long LINKS_COL_CANONICAL_URL = 268435456;
        public const long LINKS_COL_HTTP_STATUS_CODE = 536870912;
        public const long LINKS_COL_LINKS_TO_SUBDMN = 4294967296L;
        public const long LINKS_COL_LINKS_TO_ROOTDMN = 8589934592L;
        public const long LINKS_COL_ROOTDMN_LINKS_SUBDMN = 17179869184L;
        public const long LINKS_COL_PAGE_AUTHORITY = 34359738368L;
        public const long LINKS_COL_DOMAIN_AUTHORITY = 68719476736L;
    }

    public class AnchorTextConstants
    {
        public const string ANCHOR_SCOPE_PHRASE_TO_PAGE = "phrase_to_page";
        public const string ANCHOR_SCOPE_PHRASE_TO_SUBDOMAIN = "phrase_to_subdomain";
        public const string ANCHOR_SCOPE_PHRASE_TO_DOMAIN = "phrase_to_domain";
        public const string ANCHOR_SCOPE_TERM_TO_PAGE = "term_to_page";
        public const string ANCHOR_SCOPE_TERM_TO_SUBDOMAIN = "term_to_subdomain";
        public const string ANCHOR_SCOPE_TERM_TO_DOMAIN = "term_to_domain";

        public const string ANCHOR_SORT_DOMAINS_LINKING_PAGE = "domains_linking_page";

        public const long ANCHOR_COL_ALL = 0;
        public const long ANCHOR_COL_TERM_OR_PHRASE = 2;
        public const long ANCHOR_COL_INTERNAL_PAGES_LINK = 8;
        public const long ANCHOR_COL_INTERNAL_SUBDMNS_LINK = 16;
        public const long ANCHOR_COL_EXTERNAL_PAGES_LINK = 32;
        public const long ANCHOR_COL_EXTERNAL_SUBDMNS_LINK = 64;
        public const long ANCHOR_COL_EXTERNAL_ROOTDMNS_LINK = 128;
        public const long ANCHOR_COL_INTERNAL_MOZRANK = 256;
        public const long ANCHOR_COL_EXTERNAL_MOZRANK = 512;
        public const long ANCHOR_COL_FLAGS = 1024;
    }
}

```

LinksService.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Web;
using System.Web.Script.Serialization;

namespace SEOmozAPI
{
    public class LinksResponse
    {
        public string lrid;

        public string lsrc;

        public string ltgt;
    }
}

```

```

    public string lufmrp;
    public string lufmrr;
    public string lufrid;
    public string lupda;
    public string luprid;
    public string luueid;
    public string luufq;
    public string luuid;
    public string luumrp;
    public string luumrr;
    public string luupa;
    public string luupl;
    public string luus;
    public string luut;
    public string luuu;
    public string uu;
}

public class LinksService
{
    public Authenticator Authenticator
    {
        get;
        set;
    }

    public LinksService()
    {
    }

    /**
     *
     * @param authenticator
     */
    public LinksService(Authenticator authenticator)
    {
        Authenticator = authenticator;
    }

    /**
     * This method returns a set of links to a page or domain.
     */

```



```

    * @param objectURL
    * @param scope determines the scope of the Target link, as well as the Source
results.
    * @param filters filters the links returned to only include links of the
specified type. You may include one or more of the following values separated by '+'
    * @param sort determines the sorting of the links, in combination with limit and
offset, this allows fast access to the top links by several orders:
    * @param col specifies data about the source of the link is included
    * @return
    *
    * @see #GetLinks(string, string, string, string, int, int, int)
    */
    public List<LinksResponse> GetLinks(string objectURL, string scope, string
filters, string sort, long col)
    {
        return GetLinks(objectURL, scope, filters, sort, col, -1, -1);
    }

    /**
    * This method returns a set of links to a page or domain.
    *
    * @param objectURL
    * @param scope determines the scope of the Target link, as well as the Source
results.
    * @param filters filters the links returned to only include links of the
specified type. You may include one or more of the following values separated by '+'
    * @param sort determines the sorting of the links, in combination with limit and
offset, this allows fast access to the top links by several orders:
    * @param col specifies data about the source of the link is included
    * @param offset The start record of the page can be specified using the Offset
parameter
    * @param limit The size of the page can by specified using the Limit parameter.
    * @return
    */
    public List<LinksResponse> GetLinks(string objectURL, string scope, string
filters, string sort, long col, int offset, int limit)
    {
        string urlToFetch = "http://lsapi.seomoz.com/linkscope/links/" +
HttpUtility.UrlEncode(objectURL) + "?" + Authenticator.GetAuthenticationStr();

        if (scope != null)
        {
            urlToFetch = urlToFetch + "&Scope=" + scope;
        }
        if (filters != null)
        {
            urlToFetch = urlToFetch + "&Filter=" + filters;
        }
        if (sort != null)
        {
            urlToFetch = urlToFetch + "&Sort=" + sort;
        }
        if (col > 0)
        {
            urlToFetch = urlToFetch + "&SourceCols=" + col;
        }
        if (offset >= 0)
        {

```

```

        urlToFetch = urlToFetch + "&Offset=" + offset;
    }
    if (limit >= 0)
    {
        urlToFetch = urlToFetch + "&Limit=" + limit;
    }

    string response = ConnectionUtil.MakeRequest(urlToFetch);
    if (string.IsNullOrEmpty(response))
        return null;
    return new JavaScriptSerializer().Deserialize<List<LinksResponse>>(response);
}
}
}

```

URLMetricsService.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Web;
using System.Web.Script.Serialization;

namespace SEOMozAPI
{
    public class UrlResponse
    {
        /**
         * The title of the page if available. For example: "Request-Response Format"
         */
        public string ut;

        /**
         * The url of the page. For example: "apiwiki.seomoz.org/Request-Response+Format"
         */
        public string uu;

        /**
         * The subdomain of the url. For example: "apiwiki.seomoz.org"
         */
        public string ufq;

        /**
         * The root domain of the url. For example: "seomoz.org"
         */
        public string upl;

        /**
         * The number of juice-passing external links to the url.
         */
        public string ueid;

        /**

```

```

    * The number of juice-passing external links to the subdomain of the url.
    */
    public string feid;

    /**
     * The number of juice-passing external links to the root domain of the url.
     */
    public string peid;

    /**
     * The number of juice-passing links (internal or external) to the url.
     */
    public string ujid;

    /**
     * The number of subdomains with any pages linking to the url.
     */
    public string uifq;

    /**
     * The number of root domains with any pages linking to the url.
     */
    public string uipl;

    /**
     * The number of links (juice-passing or not, internal or external) to the url.
     */
    public string uid;

    /**
     * The number of subdomains with any pages linking to the subdomain of the url.
     */
    public string fid;

    /**
     * The number of root domains with any pages linking to the root domain of the
url.
     */
    public string pid;

    /**
     * The mozRank of the url. Requesting this metric will provide both the pretty
10-point score (in umrp) and the raw score (umrr).
     */
    public string umrp;

    /**
     * The mozRank of the url. Requesting this metric will provide both the pretty
10-point score (in umrp) and the raw score (umrr).
     */
    public string umrr;

    /**
     * The mozRank of the subdomain of the url. Requesting this metric will provide
both the pretty 10-point score (fmrp) and the raw score (fmrr).
     */
    public string fmrp;

```

```

/**
 * The mozRank of the subdomain of the url. Requesting this metric will provide
both the pretty 10-point score (fmrp) and the raw score (fmrr).
 */
public string fmrr;

/**
 * The mozRank of the Root Domain of the url. Requesting this metric will
provide both the pretty 10-point score (pmrp) and the raw score (pmrr).
 */
public string pmrp;

/**
 * The mozRank of the Root Domain of the url. Requesting this metric will
provide both the pretty 10-point score (pmrp) and the raw score (pmrr).
 */
public string pmrr;

/**
 * The mozTrust of the url. Requesting this metric will provide both the
pretty 10-point score (utrp) and the raw score (utrr).
 */
public string utrp;

/**
 * The mozTrust of the url. Requesting this metric will provide both the
pretty 10-point score (utrp) and the raw score (utrr).
 */
public string utrr;

/**
 * The mozTrust of the subdomain of the url. Requesting this metric will provide
both the pretty 10-point score (ftrp) and the raw score (ftrr).
 */
public string ftrp;

/**
 * The mozTrust of the subdomain of the url. Requesting this metric will provide
both the pretty 10-point score (ftrp) and the raw score (ftrr).
 */
public string ftrr;

/**
 * The mozTrust of the root domain of the url. Requesting this metric will
provide both the pretty 10-point score (ptrp) and the raw score (ptrr).
 */
public string ptrp;

/**
 * The mozTrust of the root domain of the url. Requesting this metric will
provide both the pretty 10-point score (ptrp) and the raw score (ptrr).
 */
public string ptrr;

/**
 * he portion of the url's mozRank coming from external links. Requesting this
metric will provide both the pretty 10-point score (uemrp) and the raw score (uemrr).
 */

```

```

    public string uemrp;

    /**
     * he portion of the url's mozRank coming from external links. Requesting this
    metric will provide both the pretty 10-point score (uemrp) and the raw score (uemrr).
     */
    public string uemrr;

    /**
     * The portion of the mozRank of all pages on the subdomain coming from external
    links. Requesting this metric will provide both the pretty 10-point score (fejp) and the
    raw score (fejr).
     */
    public string fejp;

    /**
     * The portion of the mozRank of all pages on the subdomain coming from external
    links. Requesting this metric will provide both the pretty 10-point score (fejp) and the
    raw score (fejr).
     */
    public string fejr;

    /**
     * The portion of the mozRank of all pages on the root domain coming from
    external links. Requesting this metric will provide both the pretty 10-point score
    (pejp) and the raw score (pejr).
     */
    public string pejp;

    /**
     * The portion of the mozRank of all pages on the root domain coming from
    external links. Requesting this metric will provide both the pretty 10-point score
    (pejp) and the raw score (pejr).
     */
    public string pejr;

    /**
     * The mozRank of all pages on the subdomain combined. Requesting this metric
    will provide both the pretty 10-point score (fjp) and the raw score (fjr).
     */
    public string fjp;

    /**
     * The mozRank of all pages on the subdomain combined. Requesting this metric
    will provide both the pretty 10-point score (fjp) and the raw score (fjr).
     */
    public string fjr;

    /**
     * The mozRank of all pages on the root domain combined. Requesting this metric
    will provide both the pretty 10-point score (pjp) and the raw score (pjr).
     */
    public string pjrp;

    /**
     * The mozRank of all pages on the root domain combined. Requesting this metric
    will provide both the pretty 10-point score (pjp) and the raw score (pjr).
     */
    public string pjrr;

```

```

public string pjr;

/**
 * If the url canaonicalizes to a different form, that canonical form will be
available in this field.
 */
public string ur;

/**
 * The HTTP status code recorded by Linkscope for this URL (if available)
 */
public string us;

/**
 * Total links (including internal and nofollow links) to the subdomain of the
url in question.
 */
public string fuid;

/**
 * Total links (including internal and nofollow links) to the root domain of the
url in question.
 */
public string puid;

/**
 * The number of root domains with at least one link to the subdomain of the url
in question.
 */
public string fipl;

/**
 * A score out of 100-points representing the likelihood for arbitrary content to
rank on this page
 */
public string upa;

/**
 * A score out of 100-points representing the likelihood for arbitrary content to
rank on this dom
 */
public string pda;

public override string ToString()
{
    return "Response [feid=" + feid + ", fejp=" + fejp + ", fejr=" + fejr
        + ", fid=" + fid + ", fipl=" + fipl + ", fjp=" + fjp + ", fjr="
        + fjr + ", fmrp=" + fmrp + ", fmrr=" + fmrr + ", ftrp=" + ftrp
        + ", ftrr=" + ftrr + ", fuid=" + fuid + ", pda=" + pda
        + ", peid=" + peid + ", pejp=" + pejp + ", pejr=" + pejr
        + ", pid=" + pid + ", pjp=" + pjp + ", pjr=" + pjr + ", pmrp="
        + pmrp + ", pmrr=" + pmrr + ", ptrp=" + ptrp + ", ptrr=" + ptrr
        + ", puid=" + puid + ", ueid=" + ueid + ", uemrp=" + uemrp
        + ", uemrr=" + uemrr + ", ufq=" + ufq + ", uid=" + uid
        + ", uifq=" + uifq + ", uipl=" + uipl + ", ujid=" + ujid
        + ", umrp=" + umrp + ", umrr=" + umrr + ", upa=" + upa
        + ", upl=" + upl + ", ur=" + ur + ", us=" + us + ", ut=" + ut
        + ", utrp=" + utrp + ", utr=" + utr + ", uu=" + uu + "];";
}

```

```

    }
}

public class URLMetricsService
{
    public Authenticator Authenticator
    {
        get;
        set;
    }

    public URLMetricsService()
    {
    }

    /**
     *
     * @param authenticator
     */
    public URLMetricsService(Authenticator authenticator)
    {
        this.Authenticator = authenticator;
    }

    /**
     *
     * This method returns the metrics about a URL or set of URLs.
     *
     * @param objectURL
     * @param col This field filters the data to get only specific columns
     *             col = 0 fetches all the data
     * @return
     */
    public UrlResponse GetUrlMetrics(string objectURL, long col)
    {
        string urlToFetch = "http://lsapi.seomoz.com/linkscope/url-metrics/" +
        HttpUtility.UrlEncode(objectURL) + "?" + Authenticator.GetAuthenticationStr();
        //System.out.println(urlToFetch);
        if (col > 0)
        {
            urlToFetch = urlToFetch + "&Cols=" + col;
        }
        string response = ConnectionUtil.MakeRequest(urlToFetch);
        return new JavaScriptSerializer().Deserialize<UrlResponse>(response);
    }

    /**
     *
     * Fetch all the Url Metrics for the objectURL
     *
     * @param objectURL
     * @return
     *
     * @see URLMetricsService#getUrlMetrics(string, int)
     */
}

```

```

        public UrlResponse GetUrlMetrics(string objectURL)
        {
            return GetUrlMetrics(objectURL, 0);
        }
    }
}

```

ProjectTest

GoogleService.asmx

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Services;
using System.Net;
using System.Text.RegularExpressions;
using System.Text;
using System.IO;
using System.Web.Script.Serialization;
using SEOmozAPI;

namespace ProjectTest
{
    public class SearchResult
    {
        public string title { get; set; }
        public string url { get; set; }
        public string pageTotalLinks { get; set; }
        public string pageExternallinks { get; set; }
        public string pageAuthority { get; set; }
        public string domainAuthority { get; set; }
        public string domainTotalLinks { get; set; }
        public string domainExternallinks { get; set; }
        public string pageMozRank { get; set; }
        public string domainMozRank { get; set; }
        public string pageMozTrust { get; set; }
        public string domainMozTrust { get; set; }
        public int anchorLinks { get; set; }
    }

    [WebService(Namespace = "http://tempuri.org/")]
    [WebServiceBinding(ConformsTo = WsiProfiles.BasicProfile1_1)]
    [System.ComponentModel.ToolboxItem(false)]
    // To allow this Web Service to be called from script, using ASP.NET AJAX, uncomment
    the following line.
    [System.Web.Script.Services.ScriptService]
    public class GoogleService : System.Web.Services.WebService

```



```

{
    private Authenticator seoMozAuthenticator;
    private URLMetricsService urlMetricsService;
    private LinksService linkMetricsService;
    private AnchorTextService anchorTextService;

    public GoogleService()
    {
        //Requests authentication from SEOmoz API, set to expire in 2500 seconds.
        seoMozAuthenticator = new Authenticator("member-id", "secret-key", 2500);
        //Creates a new URLMetricsService from SEOmoz API, to pull URL Metrics
Requests
        urlMetricsService = new URLMetricsService(seoMozAuthenticator);
        linkMetricsService = new LinksService(seoMozAuthenticator);
        anchorTextService = new AnchorTextService(seoMozAuthenticator);
    }

    // Handles the search request
    // <param name="key">the query string to search</param>
    // <param name="count">the number of results that will be returned.</param>
    [WebMethod]

    public string Search(string key, int count, string userUrl)
    {
        /**
         Search works by creating a request queue of searches and looping through
         the queue until it is empty
        **/

        //request queue
        Queue<string> requestQueue = new Queue<string>();
        //return value
        List<SearchResult> resultList = new List<SearchResult>();
        //searched url to avoid duplicate search
        int currentPage = 1, currentCount = 0;
        //sets the requestUrl to Google search format
        string requestUrl = string.Format("http://www.google.com/search?hl=en&ie=UTF-
8&oe=UTF-8&q={0}", key);
        string content;

        //enqueue the first request to the request queue.
        requestQueue.Enqueue(requestUrl);

        //this loop requests the web request with given url, parse it and extracts
        site links & page links.
        //you will get google page content by calling
        ConnectionUtil.MakeRequest(requestUrl);
        while (requestQueue.Count > 0)
        {
            //dequeue request url from queue
            requestUrl = requestQueue.Dequeue();
            //get page content for given url
            content = ConnectionUtil.MakeRequest(requestUrl);
            //extract the links from the page content
            ExtractLinks(content, ref currentCount, count, resultList, key);
            if(currentCount < count)
                FindNextPage(content, ref currentPage, requestQueue);
        }
    }
}

```

```

//before we return, we must do the users url!

//make result data
SearchResult result = new SearchResult();

//get title
result.title = getTitle(userUrl);
//get url
result.url = userUrl;

//if the URL starts like 'http://www.url.com' trim to 'www.url.com'
if (result.url.StartsWith("http://"))
    result.url = result.url.Substring(7);

//get metrics
result = getURLMetrics(result);

//wait .5 seconds between api calls
System.Threading.Thread.Sleep(500);
result.anchorLinks = getAnchorLinks(result.url, key);

//wait .5 seconds between api calls
System.Threading.Thread.Sleep(500);

resultList.Add(result);

return new JavaScriptSerializer().Serialize(resultList);
}

/// <summary>
/// extract the next page links from the google page
/// </summary>
/// <param name="content">page content to be extracted</param>
/// <param name="currentPage">the current number of google page</param>
/// <param name="requestQueue">the web request queue</param>
private void FindNextPage(string content, ref int currentPage, Queue<string>
requestQueue)
{
    /**
    Find all strings like <a* class=fl>1</a>, <a* class=fl>2</a>, <a*
class=fl>3</a>

    */

    MatchCollection matches = Regex.Matches(content, "<a.[^<>]*? class=fl>.*?[0-
9]</a>");
    string url;

    foreach (Match match in matches)
    {
        //get target link
        url = GetAttribute(match.Value, "href");

        //if link is next page link

```

```

        if (url.StartsWith("/search?"))
        {
            //get page number of link
            int pageNum = Convert.ToInt32(GetTagContent(match.Value));
            //if page number is smaller than current page number, doesn't search.
            if (pageNum > currentPage)
            {
                //set current page number to this page number
                currentPage = pageNum;
                //request google search for this page
                requestQueue.Enqueue("http://www.google.com" +
url.Replace("&", "&"));
            }
        }
    }

    // extracts the title, link from the page contents
    // <param name="content">page content to be extracted</param>
    // <param name="currentCount">the current number of result, this is changed after
processing this function</param>
    // <param name="maxCount">the final number of which result will be returned, it
is equal to count parameter of Search function</param>
    // <param name="list">the list where the result is stored</param>
    private void ExtractLinks(string content, ref int currentCount, int maxCount,
List<SearchResult> list, string key)
    {
        //get 'a' tags like <a*class=l*> to extract the links.
        MatchCollection matches = Regex.Matches(content, "<a.[^<]*?
class=l.*?>");

        foreach (Match match in matches)
        {
            //increase the result count
            currentCount++;
            //if current result counts reached to the final number of result, exit
processing.
            if (currentCount > maxCount)
                return;

            //make result data
            SearchResult result = new SearchResult();
            //get title
            result.title = GetTagContent(match.Value);
            //get url
            result.url = GetAttribute(match.Value, "href");

            //filter out youtube results(for testing reasons, and for logical
reasons)
            if (result.url.IndexOf("youtube.com") != -1)
            {
                currentCount--;
                continue;
            }

            //if the URL starts like 'http://www.url.com' trim to 'www.url.com'
            if (result.url.StartsWith("http://"))
                result.url = result.url.Substring(7);
        }
    }
}

```

```

        //get metrics
        result = getURLMetrics(result);

        //wait .5 seconds between api calls
        System.Threading.Thread.Sleep(500);
        result.anchorLinks = getAnchorLinks(result.url, key);

        //wait .5 seconds between api calls
        System.Threading.Thread.Sleep(500);

        list.Add(result);
    }
}

// extracts the URL from the content, so it can be stored in the search result
// i.e. <a href="www.example.com">Example Title</a> this will return
www.example.com
private string GetAttribute(string content, string attrName)
{
    int iStart, iEnd;

    //ex. change attrName from href to href=" so that it can be easy to find
    attribute(<a href="/search?">).
    attrName += "=\"";
    //find string
    iStart = content.IndexOf(attrName);
    //if doesn't exist, return empty result
    if (iStart == -1)
        return null;

    //if exists, extracts result
    iStart += attrName.Length;
    //find last character "<a href="/search?">
    iEnd = content.IndexOf("\"", iStart);

    //extract the link, and return it.
    return content.Substring(iStart, iEnd - iStart);
}

// extracts the title
// i.e. <a href="www.example.com">Example Title</a> this will return Example
Title
private string GetTagContent(string content)
{
    //find all strings like ">Example Title<"
    MatchCollection matches = Regex.Matches(content, ">.*?<");
    StringBuilder strBuilder = new StringBuilder();

    //extracts the text
    foreach (Match m in matches)
    {
        strBuilder.Append(m.Value.Substring(1, m.Value.Length - 2).Trim());
    }
    return strBuilder.ToString();
}

```

```

private SearchResult getURLMetrics(SearchResult data)
{
    try
    {
        //string for all the metrics you would like...each metric is +d to the
next one
        UrlResponse urlResponse = urlMetricsService.GetUrlMetrics(data.url,
URLMetricsConstants.URLMETRICS_COL_LINKS +
URLMetricsConstants.URLMETRICS_COL_EXTERNAL_LINKS
        + URLMetricsConstants.URLMETRICS_COL_MOZRANK +
URLMetricsConstants.URLMETRICS_COL_MOZTRUST +
URLMetricsConstants.URLMETRICS_COL_PAGE_AUTHORITY
        + URLMetricsConstants.URLMETRICS_COL_DOMAIN_AUTHORITY +
URLMetricsConstants.URLMETRICS_COL_LINKS_TO_ROOTDMN +
URLMetricsConstants.URLMETRICS_COL_ROOTDMN_EXTERNAL_LINKS
        + URLMetricsConstants.URLMETRICS_COL_ROOTDMN_MOZRANK +
URLMetricsConstants.URLMETRICS_COL_ROOTDMN_MOZTRUST);

        data.pageTotalLinks = urlResponse.uid;
        data.pageExternalLinks = urlResponse.ueid;
        data.pageMozRank = urlResponse.ump;
        data.pageMozTrust = urlResponse.utrp;
        data.pageAuthority = urlResponse.upa;
        data.domainAuthority = urlResponse.pda;
        data.domainTotalLinks = urlResponse.puid;
        data.domainExternalLinks = urlResponse.peid;
        data.domainMozRank = urlResponse.pmp;
        data.domainMozTrust = urlResponse.ptrp;

    }
    catch (NullReferenceException)
    {
        //if you get an error, wait 10 seconds before retrying
        System.Threading.Thread.Sleep(10000);
        getURLMetrics(data);
    }
    return data;
}

private int getAnchorLinks(string url, string key, int count = 0)
{
    int anchorLinks = 0;
    try
    {
        List<AnchorTextResponse> anchorResponse =
anchorTextService.GetAnchorText(url, AnchorTextConstants.ANCHOR_SCOPE_PHRASE_TO_PAGE,
AnchorTextConstants.ANCHOR_SORT_DOMAINS_LINKING_PAGE,
AnchorTextConstants.ANCHOR_COL_TERM_OR_PHRASE +
AnchorTextConstants.ANCHOR_COL_EXTERNAL_PAGES_LINK);

        foreach (AnchorTextResponse response in anchorResponse)
        {
            //if the anchor phrase CONTAINS the key
            if (response.aput.IndexOf(key) != -1)
                anchorLinks += Convert.ToInt16(response.apueu);
        }
    }
}

```

```

    }
    catch (NullReferenceException)
    {
        //if you get an error, wait 10 seconds before retrying
        System.Threading.Thread.Sleep(10000);
        count++;

        //if we've failed 10 times
        if (count == 10)
            return 0;
        getAnchorLinks(url, key, count);
    }

    return anchorLinks;
}

private string getTitle(string url)
{
    string title = "";
    try
    {
        title = urlMetricsService.GetUrlMetrics(url,
URLMetricsConstants.URLMETRICS_COL_TITLE).ut;
    }
    catch (NullReferenceException)
    {
        //if you get an error, wait 10 seconds before retrying
        System.Threading.Thread.Sleep(10000);
        getTitle(url);
    }

    return title;
}

}

}

```

Test.aspx

```

<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Test.aspx.cs"
Inherits="ProjectTest.Test" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head ID="head1" >
    <title>The Google Abuser</title>
    <script type="text/javascript" src="Scripts/jquery.js"></script>
    <script type="text/javascript" src="Scripts/jquery.min.js"></script>
    <script type="text/javascript" src="Scripts/jquery.flot.js"></script>

```

```

<script type="text/javascript" src="Scripts/excanvas.js"></script>

<link rel="stylesheet" type="text/css" href="Styles/style.css" />

<script type="text/javascript">
/**
Setup the variables below:
titleLength = how long the title can be before being trimmed
urlLength = how long the URL can be before being trimmed
numLength = number of decimal places for the number being trimmed
**/
var titleLength = 20;
var urlLength = 40;
var numLength = 3;

//we use the mean value of the data in many different methods, it's easier and
cleaner to keep a global array of it
var meanArray = new Array();

var isCallingAjax = false;

//enable or disable input controls
function enableControls(isEnable) {
    if (isEnable) {
        $('#searchKey').removeAttr('disabled');
        $('#searchCount').removeAttr('disabled');
        $('#searchBtn').removeAttr('disabled');
        $('#userUrl').removeAttr('disabled');
    } else {
        $('#searchKey').attr('disabled', 'disabled');
        $('#searchCount').attr('disabled', 'disabled');
        $('#searchBtn').attr('disabled', 'disabled');
        $('#userUrl').attr('disabled', 'disabled')
    }
}

//search function executes when user click 'search' button or hits enter
function search() {
    //get query string
    var key = $('#searchKey').val();
    var userUrl = $('#userUrl').val();

    //if query string is empty, alert user with error
    if (key == null || key == '') {
        alert('Please enter a keyword!');
        return;
    }

    if (userUrl == null || userUrl == '') {
        alert('Please enter your site!');
        return;
    }

    //see how many results the user wants
    var count = $('#searchCount').val();
    if (count < 0) {
        alert('Please insert number between 1~100');
    }
}

```

```

        //disable all controls to prevent clicking while searching
        enableControls(false);
        //show wait message for searching
        $('#searchFrame').show();
        //clear the results list
        $('#resultTable').html('');
        //hide the graph
        $('#graphFrame').hide();

        //request google search
        ProjectTest.GoogleService.Search(key, count, userUrl, onAjaxSuccess,
onAjaxError);
    }

    //executes when user press 'enter' as an alternative to clicking 'search'
    function onSearchKeyDown(event) {
        if (event.keyCode == 13)
            search();
    }

    /**
    Make large text to the fixed string with given length, ex. 'This is a incredibly
long title that serves no purpose'
    to 'This is an incredibly long title...' fixlen passed values are assigned in
initial setup
    */
    function getFixedString(toFixed, fixLen) {
        if (toFixed.length > fixLen)
            return toFixed.substr(0, fixLen) + "...";
        return toFixed;
    }

    //cuts numbers to certain accuracy of decimal places
    function getFixedNum(toFixed, fixLen) {
        if (toFixed.length > fixLen) {
            fixLen += (toFixed.indexOf('.') + 1);
            return toFixed.substr(0, fixLen);
        }
        return toFixed;
    }

    //pull results from array, and display in a pretty format
    //results will be displayed as: '<li><div class="title">GoogleSearch</div><a
class="url" href="http://www.url.com/">http://www.url.com/</a> class="total-
links"><div>####</div></li>'
    function getRawDataHtmlCode(entryArray) {
        var htmlCode = '';
        //loops through the array pulling raw data and building a table
        for (var i = 0; i < entryArray.length; i++) {

            //if this is the last loop, display "Your Site"
            if ((i + 1) == entryArray.length)
                htmlCode += '<tr><td class="position"><b>Your Site</b></td>';
            else
                htmlCode += '<tr><td class="position">' + (i + 1) + '</td>';
        }
    }

```



```

        htmlCode += '<td class="title">' + getFixedString(entryArray[i].title,
titleLength) + '</td>';
        htmlCode += '<td class="url"><a href="' + entryArray[i].url + '">' +
getFixedString(entryArray[i].url, urlLength) + '</a></td>';
        htmlCode += '<td class="total-links">' + entryArray[i].pageTotalLinks +
'</td>';
        htmlCode += '<td class="external-links">' +
entryArray[i].pageExternalLinks + '</td>';
        htmlCode += '<td class="page-authority">' +
getFixedNum(entryArray[i].pageAuthority, numLength) + '</td>';
        htmlCode += '<td class="page-mozRank">' +
getFixedNum(entryArray[i].pageMozRank, numLength) + '</td>';
        htmlCode += '<td class="page-mozTrust">' +
getFixedNum(entryArray[i].pageMozTrust, numLength) + '</td>';
        htmlCode += '<td class="domain-authority">' +
getFixedNum(entryArray[i].domainAuthority, numLength) + '</td>';
        htmlCode += '<td class="domain-total-links">' +
entryArray[i].domainTotalLinks + '</td>';
        htmlCode += '<td class="domain-external-links">' +
entryArray[i].domainExternalLinks + '</td>';
        htmlCode += '<td class="domain-mozRank">' +
getFixedNum(entryArray[i].domainMozRank, numLength) + '</td>';
        htmlCode += '<td class="domain-mozTrust">' +
getFixedNum(entryArray[i].domainMozTrust, numLength) + '</td>';
        htmlCode += '<td class="anchor-links">' + entryArray[i].anchorLinks +
'</td></tr>';
    }
    return htmlCode;
}

```

```

function getStatisticsHtmlCode(entryArray) {
    var pageTotalLinksArray = new Array();
    var pageExternalLinksArray = new Array();
    var pageAuthorityArray = new Array();
    var pageMozRankArray = new Array();
    var pageMozTrustArray = new Array();
    var domainAuthorityArray = new Array();
    var domainTotalLinksArray = new Array();
    var domainExternalLinksArray = new Array();
    var domainMozRankArray = new Array();
    var domainMozTrustArray = new Array();
    var anchorLinksArray = new Array();
}

```

```

//first, we build arrays to pass to the functions(length-1 to avoid
calculating our site!)
for (var i = 0; i < entryArray.length-1; i++) {
    pageTotalLinksArray[i] = (entryArray[i].pageTotalLinks - 0);
    pageExternalLinksArray[i] = (entryArray[i].pageExternalLinks - 0);
    pageAuthorityArray[i] = (entryArray[i].pageAuthority - 0);
    pageMozRankArray[i] = (entryArray[i].pageMozRank - 0);
    pageMozTrustArray[i] = (entryArray[i].pageMozTrust - 0);
    domainAuthorityArray[i] = (entryArray[i].domainAuthority - 0);
    domainTotalLinksArray[i] = (entryArray[i].domainTotalLinks - 0);
    domainExternalLinksArray[i] = (entryArray[i].domainExternalLinks - 0);
    domainMozRankArray[i] = (entryArray[i].domainMozRank - 0);
    domainMozTrustArray[i] = (entryArray[i].domainMozTrust - 0);
    anchorLinksArray[i] = (entryArray[i].anchorLinks - 0);
}

```

```

    }

    //mean must be individually calculated for each of these arrays..so we build
the global array
    meanArray[0] = getFixedNum(calculateMean(pageTotalLinksArray), numLength);
    meanArray[1] = getFixedNum(calculateMean(pageExternalLinksArray), numLength);
    meanArray[2] = getFixedNum(calculateMean(pageAuthorityArray), numLength);
    meanArray[3] = getFixedNum(calculateMean(pageMozRankArray), numLength);
    meanArray[4] = getFixedNum(calculateMean(pageMozTrustArray), numLength);
    meanArray[5] = getFixedNum(calculateMean(domainAuthorityArray), numLength);
    meanArray[6] = getFixedNum(calculateMean(domainTotalLinksArray), numLength);
    meanArray[7] = getFixedNum(calculateMean(domainExternalLinksArray),
numLength);
    meanArray[8] = getFixedNum(calculateMean(domainMozRankArray), numLength);
    meanArray[9] = getFixedNum(calculateMean(domainMozTrustArray), numLength);
    meanArray[10] = getFixedNum(calculateMean(anchorLinksArray), numLength);

    var htmlCode = '<tr><td class="rank"></td><td class="title"></td>';
    htmlCode += '<td class="url"><b>Mean</b></td>';
    htmlCode += '<td class="total-links">' + meanArray[0] + '</td>';
    htmlCode += '<td class="external-links">' + meanArray[1] + '</td>';
    htmlCode += '<td class="page-authority">' + meanArray[2] + '</td>';
    htmlCode += '<td class="page-mozRank">' + meanArray[3] + '</td>';
    htmlCode += '<td class="page-mozTrust">' + meanArray[4] + '</td>';
    htmlCode += '<td class="domain-authority">' + meanArray[5] + '</td>';
    htmlCode += '<td class="domain-total-links">' + meanArray[6] + '</td>';
    htmlCode += '<td class="domain-external-links">' + meanArray[7] + '</td>';
    htmlCode += '<td class="domain-mozRank">' + meanArray[8] + '</td>';
    htmlCode += '<td class="domain-mozTrust">' + meanArray[9] + '</td>';
    htmlCode += '<td class="anchor-links">' + meanArray[10] + '</td></tr>';

    //correlations
    htmlCode += '<tr><td class="rank"></td><td class="title"></td>';
    htmlCode += '<td class="url"><b>Correlation</b></td>';
    htmlCode += '<td class="total-links">' +
getFixedNum(calculateCorrelation(pageTotalLinksArray), numLength) + '</td>';
    htmlCode += '<td class="external-links">' +
getFixedNum(calculateCorrelation(pageExternalLinksArray), numLength) + '</td>';
    htmlCode += '<td class="page-authority">' +
getFixedNum(calculateCorrelation(pageAuthorityArray), numLength) + '</td>';
    htmlCode += '<td class="page-mozRank">' +
getFixedNum(calculateCorrelation(pageMozRankArray), numLength) + '</td>';
    htmlCode += '<td class="page-mozTrust">' +
getFixedNum(calculateCorrelation(pageMozTrustArray), numLength) + '</td>';
    htmlCode += '<td class="domain-authority">' +
getFixedNum(calculateCorrelation(domainAuthorityArray), numLength) + '</td>';
    htmlCode += '<td class="domain-total-links">' +
getFixedNum(calculateCorrelation(domainTotalLinksArray), numLength) + '</td>';
    htmlCode += '<td class="domain-external-links">' +
getFixedNum(calculateCorrelation(domainExternalLinksArray), numLength) + '</td>';
    htmlCode += '<td class="domain-mozRank">' +
getFixedNum(calculateCorrelation(domainMozRankArray), numLength) + '</td>';
    htmlCode += '<td class="domain-mozTrust">' +
getFixedNum(calculateCorrelation(domainMozTrustArray), numLength) + '</td>';
    htmlCode += '<td class="anchor-links">' +
getFixedNum(calculateCorrelation(anchorLinksArray), numLength) + '</td></tr>';
    return htmlCode;

```

```

}

function calculateMean(dataArray) {
    var mean = 0;
    for (var i = 0; i < dataArray.length; i++) {
        mean += (dataArray[i] - 0);
    }
    mean = (mean / dataArray.length);

    //must convert to string for posting
    return mean.toString();
}

function calculateCorrelation(dataArray) {

    var correlation;

    var zScoreX = 0;
    var sumOfSquaresX = 0;
    var stdDevX = 0;
    var meanX = 0;

    var meanY = 0;
    var sumOfSquaresY = 0;
    var stdDevY = 0;
    var zScoreY = 0;

    var sumOfZScores = 0;

    //calculate mean for x values
    for (var i = 0; i < dataArray.length; i++) {
        meanX += (dataArray[i] - 0);
    }
    meanX = meanX / dataArray.length;

    //calculate sum of squares for X using mean
    for (var i = 0; i < dataArray.length; i++) {
        sumOfSquaresX += Math.pow(dataArray[i] - meanX, 2);
    }

    //calculate mean for y values
    for (var i = 0; i < dataArray.length; i++) {
        meanY += (i + 1);
    }
    meanY = meanY / dataArray.length;

    //calculate sum of squares for Y using mean
    for (var i = 0; i < dataArray.length; i++) {
        sumOfSquaresY += Math.pow((i + 1) - meanY, 2);
    }

    //calculate standard deviations of x and y
    stdDevX = Math.sqrt(sumOfSquaresX / dataArray.length);
    stdDevY = Math.sqrt(sumOfSquaresY / dataArray.length);

    //calculate z scores for x and y, and the summation of them(summation of Zx *
    zy)

```

```

    for (var i = 0; i < dataArray.length; i++) {
        zScoreX = ((dataArray[i] - meanX) / stdDevX);
        zScoreY = (((i + 1) - meanY) / stdDevY);
        sumOfZScores += (zScoreX * zScoreY);
    }

    //divide by number of entries to convert to pearsons correlation coefficient
    correlation = (sumOfZScores / dataArray.length);

    //must convert to string for posting
    return correlation.toString();
}

function getWeaknesses(dataArray) {

    var weaknessArray = new Array();

    //we push the name and value of the mean divided by the
    weaknessArray.push({ name: 'total number of links to your page.', value:
((dataArray[dataArray.length - 1].pageTotalLinks / meanArray[0]) - 1) });
    weaknessArray.push({ name: 'total number of external links to your page.',
value: ((dataArray[dataArray.length - 1].pageExternalLinks / meanArray[1]) - 1) });
    weaknessArray.push({ name: 'page authority of your page.', value:
((dataArray[dataArray.length - 1].pageAuthority / meanArray[2]) - 1) });
    weaknessArray.push({ name: 'mozRank of your page.', value:
((dataArray[dataArray.length - 1].pageMozRank / meanArray[3]) - 1) });
    weaknessArray.push({ name: 'mozTrust of your page.', value:
((dataArray[dataArray.length - 1].pageMozTrust / meanArray[4]) - 1) });
    weaknessArray.push({ name: 'domain authority of your domain.', value:
((dataArray[dataArray.length - 1].domainAuthority / meanArray[5]) - 1) });
    weaknessArray.push({ name: 'total number of links to your domain.', value:
((dataArray[dataArray.length - 1].domainTotalLinks / meanArray[6]) - 1) });
    weaknessArray.push({ name: 'total number of external links to your domain.',
value: ((dataArray[dataArray.length - 1].domainExternalLinks / meanArray[7]) - 1) });
    weaknessArray.push({ name: 'mozRank of your domain.', value:
((dataArray[dataArray.length - 1].domainMozRank / meanArray[8]) - 1) });
    weaknessArray.push({ name: 'mozTrust of your domain.', value:
((dataArray[dataArray.length - 1].domainMozTrust / meanArray[9]) - 1) });
    weaknessArray.push({ name: 'total number of links containing your targeted
keyword.', value: ((dataArray[dataArray.length - 1].anchorLinks / meanArray[10]) - 1) });

    weaknessArray.sort(sortNumber);

    var htmlCode = '';

    htmlCode += 'Your biggest weakness is the ' + weaknessArray[0].name + ' To
increase rankings, you should the increase the ' + weaknessArray[0].name;
    htmlCode += '<br /><br />Your second biggest weakness is the ' +
weaknessArray[1].name + ' To increase rankings, you should increase the ' +
weaknessArray[1].name;
    htmlCode += '<br /><br />Your third biggest weakness is the ' +
weaknessArray[2].name + ' To increase rankings, you should increase the ' +
weaknessArray[2].name;

    return htmlCode;
}

```

```

//custom sort function needed to sort array by values
function sortNumber(a, b) {
    return a.value - b.value;
}

function createExcelSheet() {
    var x = resultTable.rows

    var xls = new ActiveXObject("Excel.Application")
    xls.visible = true
    xls.Workbooks.Add

    //loops through the cells and adds the data to the cells in excel
    for (i = 0; i < x.length; i++) {
        var y = x[i].cells

        for (j = 0; j < y.length; j++)
        {
            xls.Cells(i + 1, j + 1).Value = y[j].innerText
        }
    }
}

function buildGraph(entryArray) {
    var d1 = [];
    var d2 = [];
    var d3 = [];
    var d4 = [];

    //push variables onto the array, x value is position, y value is metric value
    for (var i = 0; i < entryArray.length-1; i++) {
        d1.push([(i + 1), entryArray[i].pageMozRank]);
        d2.push([(i + 1), entryArray[i].pageMozTrust]);
        d3.push([(i + 1), entryArray[i].domainMozRank]);
        d4.push([(i + 1), entryArray[i].domainMozTrust]);
    }

    //build graph, giving the data labels for the key, and the data metrics
    $.plot($("#graphFrame"), [
    { label: "Page mozRank", data: d1 },
    { label: "Page mozTrust", data: d2 },
    { label: "Domain mozRank", data: d3 },
    { label: "Domain mozTrust", data: d4 }
    ],
    {
        series: {
            points: { show: true }
        },

        //setup display options for the xaxis
        xaxis: {
            ticks: (entryArray.length-1),
            min: 0,
            max: (entryArray.length-1)
        },

        //setup display options for the yaxis

```

```

        yaxis: {
            ticks: 20,
            min: 0,
            max: 10
        },

        //stylized grid
        grid: {
            backgroundColor: { colors: ["#fff", "#eee"] }
        }
    });
}

```

//executes once the results have been pulled. This will automatically display results by using different functions to calculate html code, and then populate the different frames

```

function onAjaxSuccess(result) {

    //hide search wait message
    $('#searchFrame').hide();

    //display calculate wait message
    $('#calculateFrame').show();

    //html that will be pushed to the display
    var html = [''];

    //convert json string to object variable
    var entries = eval(result);

    //make header row for the displays
    html.push('<tr><th class="position title">Position</th><th class="title subject">Title</th><th class="url subject">URL</th><th class="total-links subject">Page Total Links</th><th class="external-links subject">Page External Links</th><th>Page Authority</th><th>Page mozRank</th><th>Page mozTrust</th><th>Domain Authority</th><th>Domain Total Links</th><th>Domain External Links</th><th>Domain mozRank</th><th>Domain mozTrust</th><th>Anchor Links</th></tr>');

    //build html codes for raw data
    html.push(getRawDataHtmlCode(entries));

    //calculate and get html for statistics html
    html.push(getStatisticsHtmlCode(entries));

    //hide calculate wait message
    $('#calculateFrame').hide();

    //insert results into the table
    $('#resultTable').html(html.join(''));

    //clear the html, and setup header for weakness frame
    html = ['<br /><br /><h2>How to Improve Your Rankings:</h2>'];

    //get the weaknesses for your site and provide suggestions for improvement
    html.push(getWeaknesses(entries));

    //push weaknesses to the weakness div
    $('#weaknessFrame').html(html.join(''));
}

```

```

        //display graph div(this must be before the buildGraph call
        $('#graphFrame').show()

        //graph the 4 metrics
        buildGraph(entries);

        //enables controls
        enableControls(true);
        $('#exportFrame').show();
    }

    //executes when if an error occurs during ajax request
    function onAjaxError(error) {
        //hide wait messages
        $('#searchFrame').hide();
        $('#calculateFrame').hide();
        //enable controls
        enableControls(true);
        alert(error);
    }
</script>
</head>
<body>
<div align=center style="margin:40px">
<br /><br />
    <form id="Form1" runat="Server">
        <!-- Make available to use WebService 'GoogleService' -->
        <asp:ScriptManager ID="ScriptManager1" runat="server" EnablePageMethods="True">
            <Services>
                <asp:ServiceReference Path="GoogleService.asmx" />
            </Services>
        </asp:ScriptManager>

        <div>
            <label>Number of Results to Pull:</label>
            <input type="text" id="searchCount" value="10" />

            <label>Your Targeted Keyword:</label>
            <input type="text" id="searchKey" onkeydown="onSearchKeyDown(event);" />

            <label>The Page You Want to Rank:</label>
            <input type="text" id="userUrl" onkeydown="onSearchKeyDown(event);"
value="www.example.com/" />

            <input type="button" id="searchBtn" value="Search" onclick="search();" />
        </div>
        <div>
            <!-- wait message will be showed while searching-->
            <div id="searchFrame" style="display:none;">
                &nbsp; Please wait while pulling data(this
may take some time)
            </div>
            <div id="calculateFrame" style="display:none;">
                &nbsp; Please wait while we perform data
calculations(this may take some time)
            </div>

```

```

<div align="center" style=" width:90%; overflow:auto;">
<br /><br />
<table id="resultTable" border="1">

</table>
</div>
<div id="exportFrame" style="display:none;">
  <input type="button" id="Button2" value="Export to Excel"
onclick="createExcelSheet();" />
</div>
<br /><br /><br />
<div id="graphFrame" style="width:600px;height:300px;"></div>

  <div id="weaknessFrame" align=left></div>
</div>
</form>

</div>
</body>
</html>

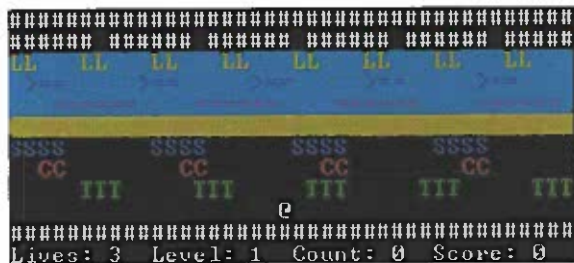
```


COIN 315: Grogger

Grade: 85

Notes: Grogger was created for COIN 315. The program was designed to replicate the classic arcade game Frogger. One of the main challenges of this program was the extensive layout with multiple files. The final project consisted of 27 separate .cpp and .h files. The program can be compiled in any C++ compiler, and running main.cpp will execute the program.

Screenshots:



main

```
#FILE: main.cpp
```

```
#include "world.h"
```

```
#Student Name:Michia Rohrssen
```

```
#Program Name:Grogger
```

```
#Creation Date:3/15/2009
```

```
#Last Modified Date:5/3/2009
```

```
#COIN Course:COIN 325
```

```
#Grade Received:85
```

```
#Comments regarding design:The entire program was designed from  
#the ground up to be dynamic in it's structure. All elements  
#of the game can be easily changed, such as size of the game,  
#character representations, speed, and lives.
```

```
void main(int argc, char *argv[] ) {  
    int delay=100;  
    if (argc==2) delay=atoi(argv[1]);  
    world w(delay);  
    w.run();  
    return;  
}
```